RAG FOR NLP

DAY 2 OF 2

Measure, improve and ship RAG

Measure whether RAG works, improve its search one change at a time, and put it in front of users safely.

TAREK ATWAN

OCT 2026

RAG FOR NLP

BY THE END OF TODAY, YOU CAN

By the end of today you can

01 Measure search and answers with the right metric

02 Build a golden set, and check an LLM judge before you trust it

03 Improve search with hybrid search and reranking, and show it helped

04 Turn a follow-up question into a search that finds the right evidence

05 Ship with permissions, release tests and monitoring

RECAP

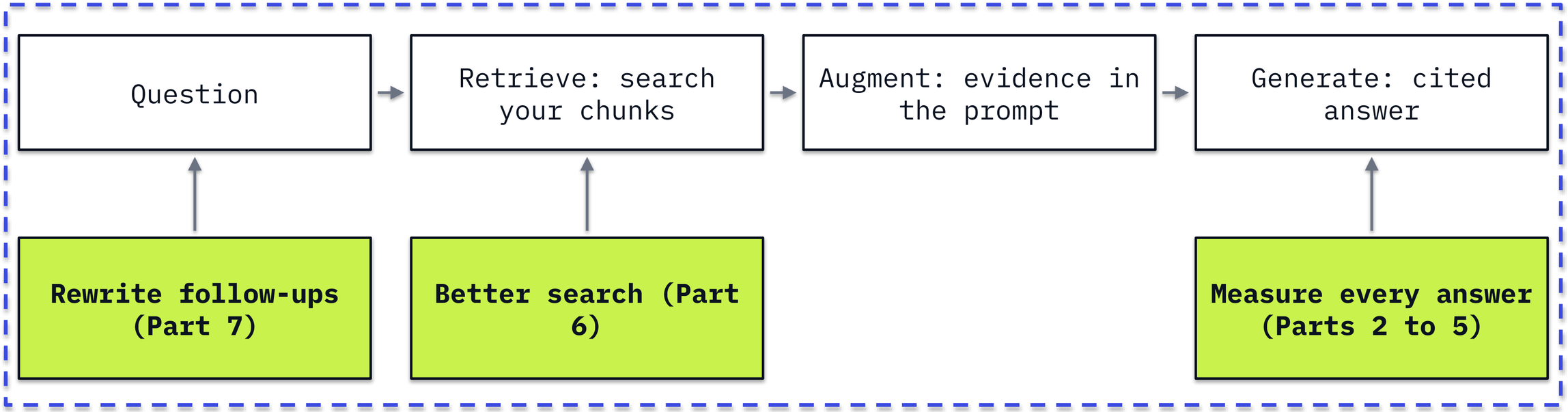
Where Day 1 left off

Yesterday you built RAG step by step and found the step that broke. Today you measure it, improve it and ship it.

DAY 1 IN ONE PICTURE

Retrieve, augment, generate, and what today adds

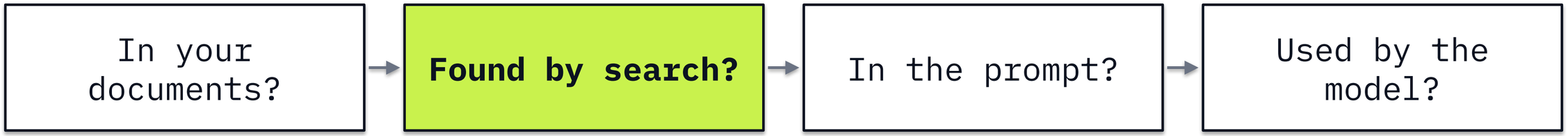
The same three steps as yesterday. Three of today's additions are in lime; the fourth, a service, goes around it all.



AROUND IT ALL: A SERVICE WITH PERMISSIONS, RELEASE TESTS AND MONITORING (PART 9)

An answer goes wrong in one of four places

When an answer is wrong, ask four questions in order. The first “no” is the step to fix.



If not: fix loading and parsing

If not: fix chunks or search

If not: fix filters or the token limit

If not: fix the prompt

TIP Day 1 found each failure by reading one trace at a time. Today we count them over many questions.

TODAY’S SHIFT

From checking by eye to scoring every question

One answer read carefully tells you what went wrong once. A fixed set of questions tells you whether a change helped.

Day 1: one answer, by eye

Read the trace for one question
Spot the step that failed
Good for finding a bug
Says nothing about the other questions

Day 2: every question, the same way

A fixed set of questions with the answers you want
The same scores after every change
Shows what a change fixed and what it broke
Then you decide what ships

PART 1

Rebuild the pipeline with a framework

Day 1 wrote every step in plain Python so you could see it. A framework gives you the same steps as ready-made pieces, with defaults you did not choose.

DEFINITION

RAG framework: a library of ready-made pieces for each RAG step, which you connect instead of writing each step yourself.

Pieces: loaders, text splitters, embeddings, vector stores, retrievers, model calls. Examples: LangChain, LlamaIndex, Haystack.

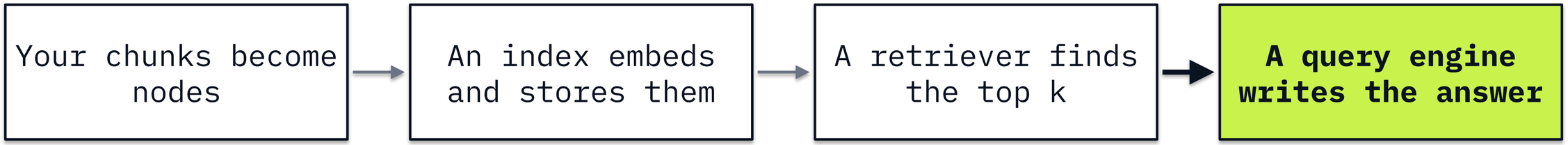
Each Day 1 step has a LangChain piece

Same pipeline, same models, same chunks. Metadata means the ID, source and other fields stored with each chunk.

DAY 1, IN PLAIN PYTHON	THE LANGCHAIN PIECE
A chunk with its ID and source	`Document: text plus metadata`
Turn text into numbers	`Embeddings: the model from your settings`
Keep vectors with their IDs	`Vector store: in memory, or a database`
Find the closest chunks	**`Retriever: returns the top k`**
Ask the model	`Chat model: the model from your settings`

LlamaIndex builds the same pipeline around an index

Made for RAG: you hand it documents and ask questions. More is decided for you, so there is more to check.



TIP The query engine wraps retrieval and the prompt in one call. Handy, until you need to see what reached the model.

EXAMPLE · FROM LAB 05 AND OPTIONAL A

A framework chooses defaults for you

Both frameworks found the right paragraph for all 30 questions, yet handed the answer step different chunks than your own code.

LangChain

Returns 4 chunks unless you ask for 3. Your Day 1 code returned 3.

Same hit@3; different evidence for every question.

LlamaIndex

Returns 2 chunks by default, and embeds each chunk's title and metadata together with its text.

Same top 3 for only about half the questions.

What to do

Set k and the other settings yourself.

Compare with your own code question by question, never by the total alone.

WATCH A total like "30 of 30" can match while every question gets different evidence.

THE OPTIONS

Three ways to build retrieval

The further right, the less code you write and the less you can see or change.

Your own code Day 1	A framework LangChain, LlamaIndex	A managed service OpenAI file search
You write: every step	You write: settings and glue	You write: uploads and calls
You see: everything	You see: most steps, if you look	You see: the results
Cost: your time	Cost: defaults to check	Cost: hidden ranker, data leaves

OPTIONAL B

A managed search service runs retrieval for you

You upload files and send questions. The service chunks, embeds, ranks and returns results; some of that you cannot see.

You can set

Chunk size and overlap
Number of results
A score threshold
Whether to rewrite the query

You cannot see

The embedding model
The vectors
How the ranker reorders results

What it means

Measure it on your own questions, as with your own search. Optional B finds part of its score comes from a ranker you cannot inspect.

INSTRUCTOR DEMO

Lab 05: Rebuild the pipeline with LangChain

Watch Day 1’s search rebuilt in LangChain and checked against your own, question by question. In class, as a demo: Parts A, C and D, with the embeddings cell at the start of Part B.

In class

10 minutes

Folder 05_Rebuild_With_LangChain

PART 2

Measure before you change anything

A change that fixes one question can break another. Write the questions down first, then score every one the same way, every time.

MADE-UP EXAMPLE

A change can fix one question and break another

Six questions, before and after a change to search. The total did not move; two questions did.

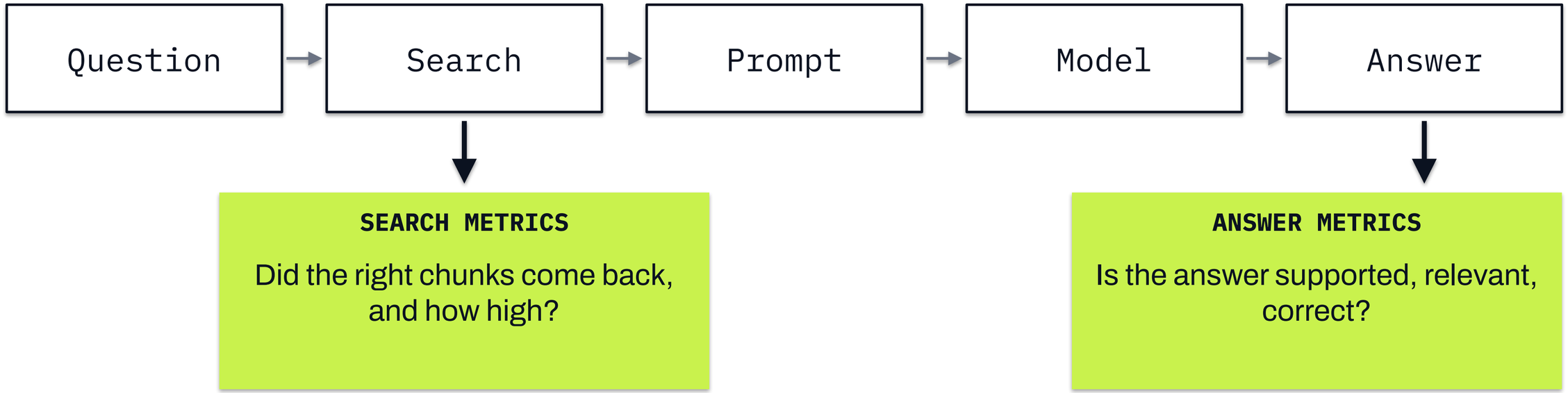
Question	About	Before	After
Q1	Atlas link expiry	pass	pass
Q2	Boreal link expiry	fail	**pass**
Q3	Atlas email reset	pass	pass
Q4	Temporary accounts	pass	fail
Q5	Report export	pass	pass
Q6	Invoices	fail	fail
Total		4 of 6	4 of 6

TIP Q2 was fixed and Q4 broke. An average hides both; a question-by-question table shows them.

WHERE TO MEASURE

Measure the search and the answer separately

If search misses, the answer fails whatever the model does. If search succeeds, the answer can still fail. Two kinds of score find which.



DEFINITION

Relevance label: a person’s note of which chunks answer a question.

Search metrics compare what search returned with these labels. For “How long is an Atlas reset link valid?” the label is H02.

MADE-UP EXAMPLE · USED ON THE NEXT FIVE PAGES

One question, five results: the list every metric reads

Question: “How do I reset my Atlas password, and how long is the link valid?” Three chunks are relevant: H01, H02 and a help note, H07, that search never returned.

WHAT SEARCH RETURNED

1	H04 · Boreal reset links expire after 10 minutes	no
2	H01 · Atlas passwords can be reset by email	**RELEVANT**
3	H05 · Atlas invoices are sent monthly	no
4	H02 · Atlas reset links expire after 30 minutes	**RELEVANT**
5	H03 · Atlas exports reports as CSV files	no

NOT RETURNED

H07 · Atlas reset emails can take five minutes to arrive.

Relevant, but outside the list.

SEARCH METRIC 1 OF 5

Hit@k: did any right chunk make the top k?

Look at the top k results. If at least one relevant chunk is there, the question scores 1; otherwise 0. Day 1's hit@3 is this, with k = 3.

THE LIST, k = 3		TOP 3
1	H04 · Boreal reset links expire after 10 minutes	no
2	H01 · Atlas passwords can be reset by email	RELEVANT
3	H05 · Atlas invoices are sent monthly	no
4	H02 · Atlas reset links expire after 30 minutes	RELEVANT
5	H03 · Atlas exports reports as CSV files	no

HIT@3 FOR THIS QUESTION

1

H01 is at rank 2

Blind to where in the top 3 the right chunk sits, and to how many right chunks came back.

SEARCH METRIC 2 OF 5

Precision@k: how much of the top k is useful

Of the k chunks search returned, the share that are relevant. Low precision fills the prompt with text that does not help.

THE LIST, k = 3		TOP 3
1	H04 · Boreal reset links expire after 10 minutes	no
2	H01 · Atlas passwords can be reset by email	RELEVANT
3	H05 · Atlas invoices are sent monthly	no
4	H02 · Atlas reset links expire after 30 minutes	RELEVANT
5	H03 · Atlas exports reports as CSV files	no

$$\text{precision@}k = \frac{\text{relevant in top } k}{k}$$

PRECISION@3

1 / 3 = 0.33

one of the top three is relevant

Recall@k: how much of what you need came back

Of all the relevant chunks, the share search returned in the top k. It needs every relevant chunk labelled, which is rare.

THE LIST, k = 3 — TOP 3

1	H04 · Boreal reset links expire after 10 minutes	no
2	H01 · Atlas passwords can be reset by email	RELEVANT
3	H05 · Atlas invoices are sent monthly	no
4	H02 · Atlas reset links expire after 30 minutes	RELEVANT
5	H03 · Atlas exports reports as CSV files	no

$$\text{recall@}k = \frac{\text{relevant in top } k}{\text{all relevant}}$$

RECALL@3

1 / 3 = 0.33

H01 found; H02 at rank 4; H07 missed

TIP Precision and recall pull against each other: return more chunks and recall rises while precision falls.

SEARCH METRIC 4 OF 5

MRR: how high the first right chunk sits

Reciprocal rank is 1 divided by the rank of the first relevant chunk. MRR is its mean over all questions; a miss scores 0 and still counts.

FIRST RELEVANT: RANK 2

1	H04 · Boreal reset links expire after 10 minutes	no
2	H01 · Atlas passwords can be reset by email	RELEVANT
3	H05 · Atlas invoices are sent monthly	no
4	H02 · Atlas reset links expire after 30 minutes	RELEVANT
5	H03 · Atlas exports reports as CSV files	no

$$\mathrm{MRR} = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{\mathrm{rank}_i}$$

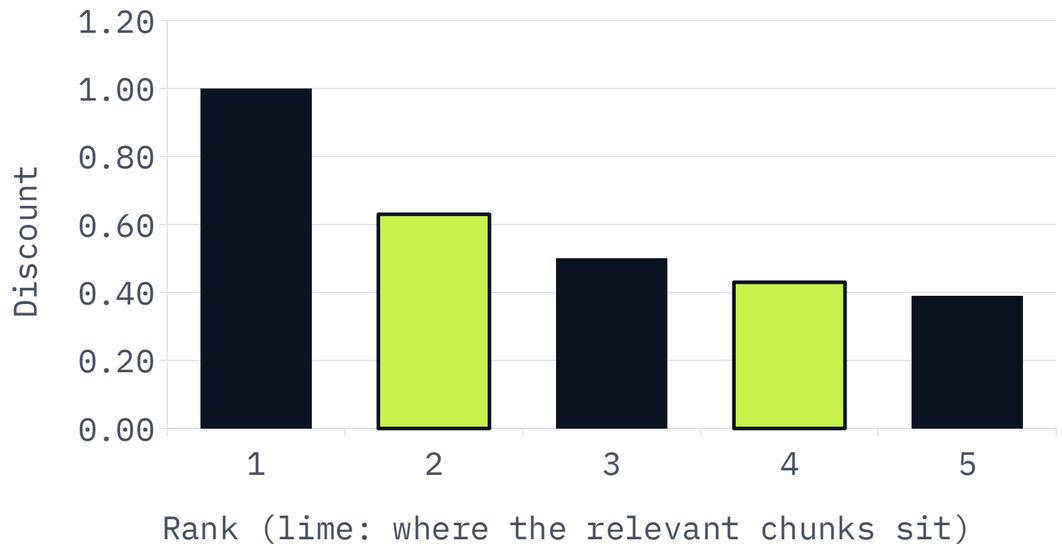
THIS QUESTION

1 / 2 = 0.50

three questions, first hit at 1, 3, none: 0.44

nDCG: credit for right chunks near the top

Each relevant chunk earns its grade, discounted by its rank; the total is divided by the best possible ordering. Grades: H02 3, H01 2, H07 1.



$$\mathrm{DCG} = \sum_{i} \frac{\mathrm{grade}_i}{\log_2(i+1)} \qquad \mathrm{nDCG} = \frac{\mathrm{DCG}}{\text{ideal DCG}}$$

THIS LIST

2.55 / 4.76 = 0.54

ideal: H02, H01, H07 at ranks 1 to 3

TIP

Grades let a perfect chunk count more than a useful one. Introduced by Järvelin and Kekäläinen (2002); this discount is today’s usual form.

Search metrics at a glance

Each answers a different question about the same list. Report the one that matches what you need, with k and the unit (chunk or document).

Metric	Answers	Use it when	Needs
hit@k	Any right chunk in the top k?	One chunk can answer	One label
precision@k	How much of the top k helps?	The prompt is small	Top-k labels
recall@k	How much of what is needed?	Answers need several chunks	All labels
MRR	How high is the first right one?	The first chunk matters most	One label
nDCG@k	Best chunks near the top?	Some chunks are better	Graded labels

TIP

Mean average precision (MAP) is another rank-aware metric. Sources: Manning, Raghavan and Schütze (2008); Hands-On RAG for Production, ch. 6.

QUICK CHECK



Search always puts the right chunk at rank 3. hit@3 is 1.0 on every question. What is MRR, and what does it tell you?

ANSWER

hit@3 is perfect; MRR is 0.33

01

MRR is 1/3 = 0.33: the first right chunk always sits at rank 3.

02

hit@3 hides it. The right evidence always arrives last, below two weaker chunks. A reranker (Part 6) targets exactly this.

PART 3

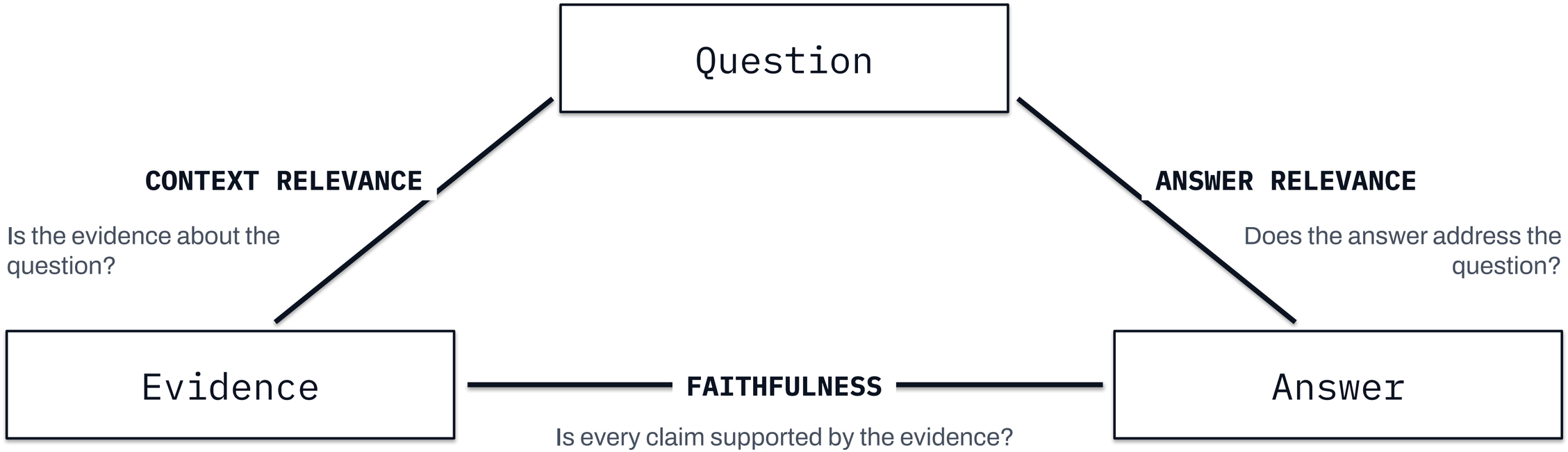
Measure the answer, not only the search

The right evidence can still produce a wrong answer. Three links between question, evidence and answer show where.

THREE CHECKS

Three checks between question, evidence and answer

Each check compares two of the three. TruLens calls this the RAG triad; Ragas and DeepEval use the same three ideas.



ANSWER CHECK 1 OF 6

Context relevance: is the evidence about the question?

Before blaming the model, check what it was given. Each chunk in the prompt is judged against the question.

QUESTION: “HOW LONG IS AN ATLAS RESET LINK VALID?”

H02 · Atlas reset links expire after 30 minutes. Request a new link if the old link has expired.	**RELEVANT**
H01 · Atlas passwords can be reset by email. Temporary accounts require a support agent.	**RELATED**
H04 · Boreal reset links expire after 10 minutes. Password reset uses a mobile app.	**WRONG PRODUCT**

CONTEXT RELEVANCE

1 of 3 chunks

H04 is the danger: it looks relevant, and it holds a different number. A model that uses it gives a wrong, cited answer.

DEFINITION

Faithfulness: every claim in the answer is supported by the evidence the model was given.

It compares the answer with the evidence, not with the truth. An answer can be faithful to an out-of-date policy and still be wrong.

ANSWER CHECK 2 OF 6 · MADE-UP ANSWER

Faithfulness is checked one claim at a time

Split the answer into claims; label each against the evidence, quoting the words it was checked against. Evidence: H01 and H02.

> “Atlas reset links last 30 minutes. You can also reset by phone. Temporary accounts may need an administrator.”

1	Links last 30 minutes · H02: “expire after 30 minutes”	**SUPPORTED**
2	You can reset by phone · no evidence mentions a phone	**NOT SUPPORTED**
3	May need an administrator · H01: “require a support agent”	**CONTRADICTED**

CLAIMS SUPPORTED

1 of 3

PASS OR FAIL

Fail: any unsupported claim

ANSWER CHECK 3 OF 6

Answer relevance: does it address what was asked?

An answer can be true, cited and faithful, and still not answer the question.

QUESTION

How long is an Atlas reset link valid?

ANSWER

Atlas passwords can be reset by email [H01].

Check	Result	Why
Faithfulness	Pass	H01 says exactly that
Answer relevance	**Fail**	It answers how to reset, not how long the link lasts

Correctness: compare with a reference answer

Exact match (EM) is 1 only if the answers are identical. Token F1 scores shared words from both sides: the answer's and the reference's. Reference: "30 minutes".

Answer	EM	Shared words	F1	Actually
"30 minutes"	1	2 of 2	1.00	Right
"about 30 minutes"	0	2 of 3	0.80	Right
"10 minutes"	0	1 of 2	**0.50**	Wrong
"Atlas reset links expire after 30 minutes."	0	2 of 7	**0.44**	Right

The wrong "10 minutes" scores 0.50; the right full sentence scores 0.44. Word overlap cannot tell whether the number is right.

TIP Other options: semantic similarity (embeddings or BERTScore), or a judge given the reference. Each needs checking too.

ANSWER CHECK 5 OF 6

Citation support: the source must say what the claim says

Day 1’s two checks, now counted over every answer: the cited ID was in the prompt, and its text supports the claim.

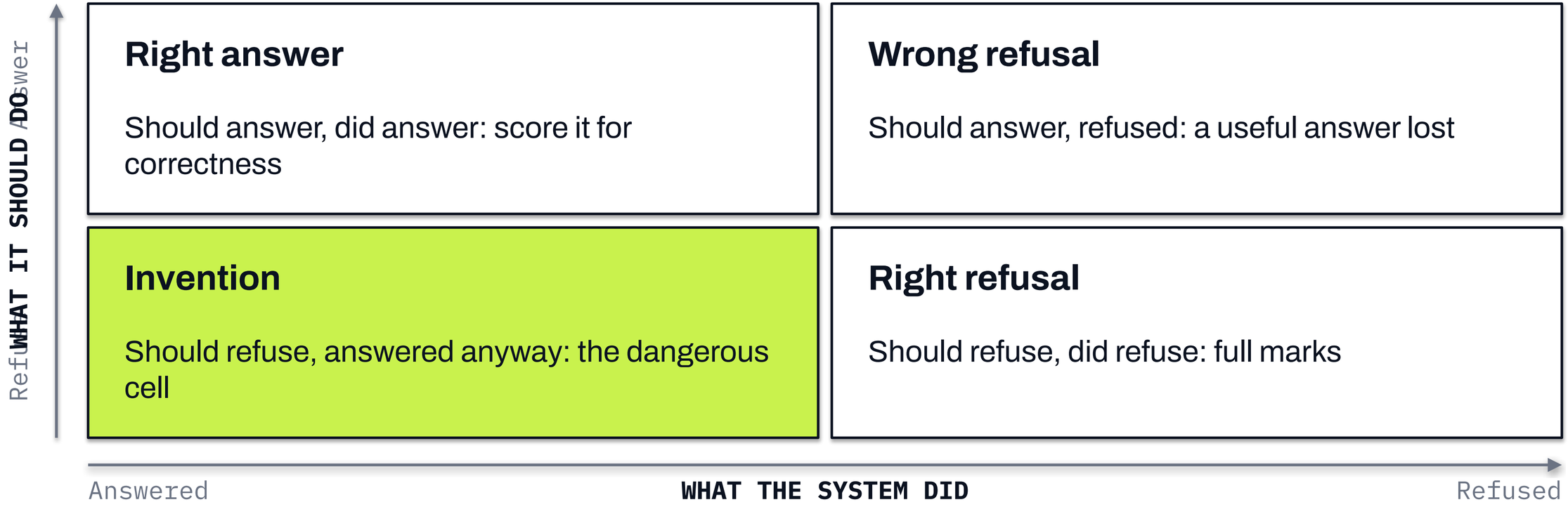
Answer	ID in the prompt?	Text supports it?	Verdict
30 minutes [H02]	Yes	Yes	Supported
10 minutes [H02]	Yes	No: H02 says 30	**Contradicted**
30 minutes [H09]	No: H09 was never given	Nothing to check	Made-up ID
30 minutes	No ID	Nothing to check	True, but uncited

TIP

Citation precision: cited claims their source supports. Citation completeness: claims that carry a supporting citation.

Refusals: score whether it refused when it should

Some questions should get “Insufficient evidence.” Score them as their own group, or a system that answers everything looks fine.



Answer checks at a glance

Each compares two things. Three need no reference answer, which is why tools can run them on live traffic.

Check	Compares	Reference?	Catches
Context relevance	Evidence ⇔ question	No	Off-topic chunks
Faithfulness	Claims ⇔ evidence	No	Invented or changed claims
Answer relevance	Answer ⇔ question	No	Answers to another question
Correctness	Answer ⇔ reference	Yes	Wrong facts, even cited
Citation support	Cited ID ⇔ its claim	No	Real IDs on wrong claims
Refusal	Behaviour ⇔ expected	Yes	Inventions, lost answers

TIP

Sources: Es et al. (2023), Ragas; TruLens's RAG triad; Hands-On RAG for Production, ch. 6.

QUICK CHECK



An old H02 said links last 24 hours; today’s says 30 minutes. The answer says 24 hours, citing the old H02. Faithful? Correct?

ANSWER

Faithful to its evidence, and still wrong

01

Faithful: the answer matches the evidence it was given.

02

Not correct: the evidence was out of date. Keeping old versions out of the index is a deployment job (Part 9).

PART 4

A fixed set of questions for every change

Write down the questions and the answers you want before you change anything. Then every change is scored on the same set.

DEFINITION

Golden set: questions with the answer you expect, or the refusal you expect, and where the answer lives, checked by a person.

Atlas link → “30 minutes [H02]”. Tomorrow’s weather in Amman → “Insufficient evidence.” Used to score every change.

ONE RECORD

What one golden record holds

One record per question. Keep the set out of the vector store you search, or search can find its own answer key.

ID	G02
Question	How long does an Atlas reset link stay valid?
Kind	answerable
Answer lives in	H02
Accepted answers	“30 minutes”, “half an hour”
Or expect	“Insufficient evidence.” for a refusal question
Version	set v1 · policies of 1 Oct 2026

Why each field

Kind: score slices.
Where it lives: score search.
Accepted answers: score correctness.
Version: what it was scored under.

KINDS OF QUESTION

Test the hard cases, not only the easy ones

Give each kind its own questions and its own score (a slice). An average over easy questions hides the hard ones.

Answerable

How long is an Atlas link valid? → 30 minutes [H02]

Paraphrase

When does the password recovery email stop working? → same answer, other words

Exception

Can a temporary account reset by email? → no, it needs a support agent [H01]

Wrong product

How long is a Boreal link valid? → 10 minutes [H04], not Atlas's 30

Out of scope

What is the weather tomorrow? → "Insufficient evidence."

Follow-up

"What about Boreal?" after an Atlas question → resolve it first (Part 7)

Where labels come from

Four sources of questions and answers. Most teams mix them: experts for the core, a model for breadth, users for what really happens.

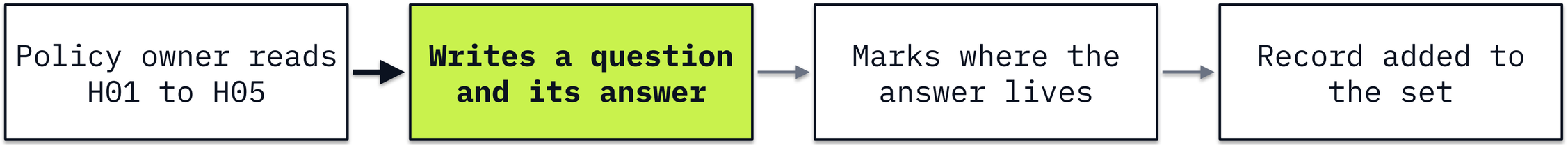
Source	Strength	Weakness
Experts	The most trusted labels	Slow and costly
Crowd workers	Many labels, quickly	Less domain knowledge; needs checks
A model (synthetic)	Fast, broad coverage	Mistakes look plausible; a person reviews
Real users	The questions people actually ask	Noisy; arrives only after release

TIP Source: Unlocking Data with Generative AI and RAG, ch. 9 (ground truth: human annotation, experts, crowdsourcing, synthetic).

LABEL SOURCE 1 OF 4

Labels from experts: slow, costly, most trusted

Someone who knows the policies reads them, writes questions a customer would ask, and the answer each should get.

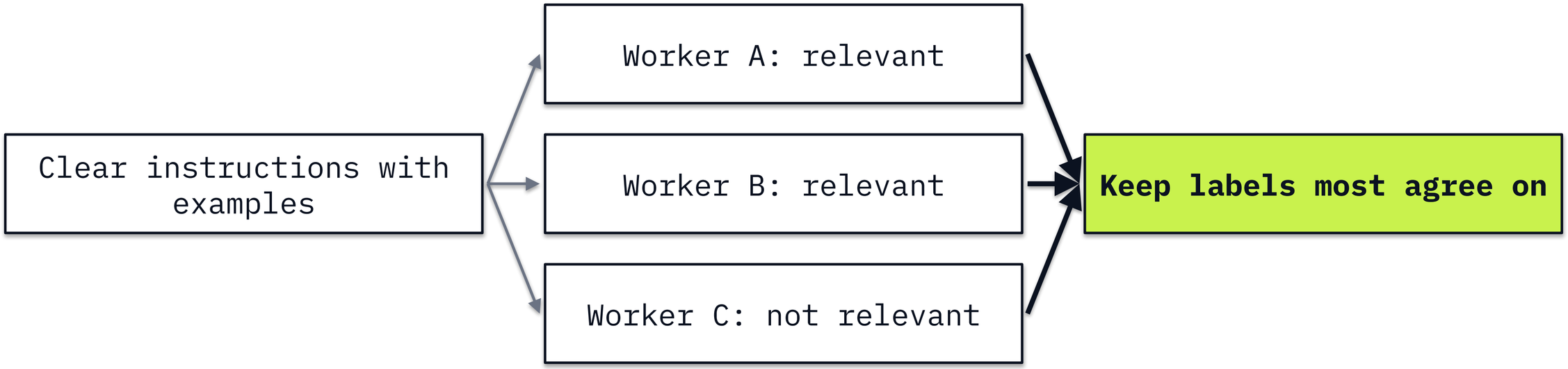


TIP Use experts for the cases that matter most: exceptions, wrong products, refusals.

LABEL SOURCE 2 OF 4

Labels from crowd workers: many labels, less expertise

Several people label each item from clear instructions; agreement between them shows which labels to trust.

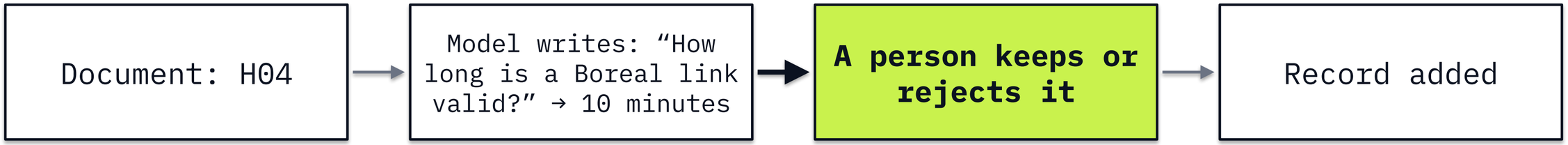


TIP Disagreement is information: it often means the question or the instruction is unclear.

LABEL SOURCE 3 OF 4

Labels from a model: fast and broad, still reviewed

A model reads each document and writes questions it answers. Ragas has a test-set generator that does this.

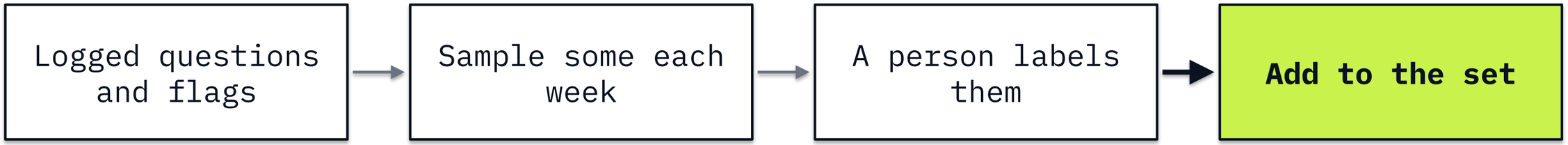


WATCH Synthetic questions tend to reuse the document’s own words, so they are easier than real ones.

LABEL SOURCE 4 OF 4

Labels from real users: the questions people ask

After release, real questions and their thumbs-down or flags show where the system fails. Sample them, label them, add them.



TIP Strip personal data before a logged question becomes a test record.

TWO SETS

Tune on one set, score once on another

Look at and tune on the development set. Score the held-out set once, at release. Tune on it and its score stops meaning anything.

TUNE	SCORE ONCE
Development set: look, change, rerun as often as you like	**Held-out set: scored once, at release**

TIP Keep questions about one document in one set, so the held-out set is not a copy of what you tuned on.

LAB TIME

Lab 06: Evaluation set

Write a golden set for your pipeline and score it one question at a time; see why a word-overlap score cannot tell a right number from a wrong one. In class: Parts A to D, except the versioning subsection at the end of Part C.

In class

30 minutes

Folder 06_Evaluation_Set

Break

Next: an LLM as judge

Let a model grade the answers, then check the grader.

PART 5

An LLM as judge, and how far to trust it

Reading every answer by hand does not scale. A second model can grade them, but its grades are model replies: check them against people.

DEFINITION

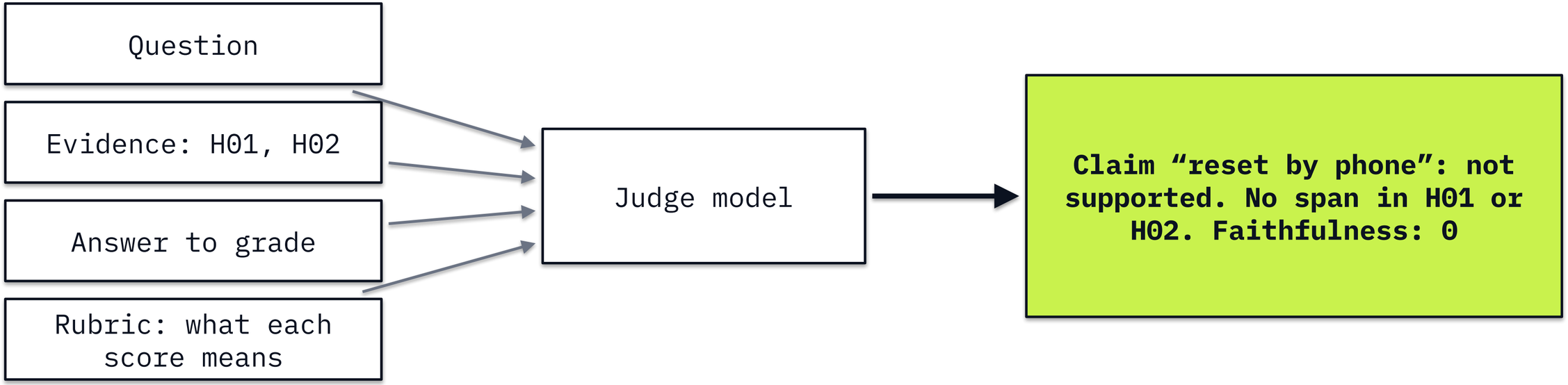
LLM as judge: a model given the question, the evidence, the answer and a written rubric, which returns a label and its reason.

A rubric is the written list of what each grade means. A judge reads meaning, which word overlap cannot, and it can be wrong.

HOW IT WORKS

What a judge reads, and what it returns

The four inputs go in as data. The reply is a label plus a reason that quotes the words it compared, so a person can check it.



WATCH Tell the judge that the evidence and the answer are data, not instructions: a planted line can try to steer the grader too.

THE RUBRIC

Write the rubric before you read any grades

Each score gets a meaning and an example. Fix it before grading starts, and give the same rubric to the judge and to people.

`Faithfulness 1`	Every claim is supported: “Links expire after 30 minutes [H02].”
`Faithfulness 0`	Any claim is unsupported: “…you can also reset by phone.”
`Faithfulness NA`	A refusal makes no claims: “Insufficient evidence.”
`Correct 2 / 1 / 0`	Matches the reference / partly / wrong or missing

TWO WAYS TO ASK

Grade one answer, or pick the better of two

Pointwise scores each answer on its own. Pairwise asks which of two is better: steadier when the judge model changes (Zheng et al., 2023), but more calls and open to position bias.

Pointwise: one answer, one score

“Score this answer 0 or 1 for faithfulness.”
Good for tracking a score over time
Scores can drift between runs

Pairwise: which of two is better?

“Answer A or answer B: which is better supported?”
Good for comparing two versions
Order can sway the verdict (next pages)

TIP

Sources: Zheng et al. (2023), Judging LLM-as-a-judge with MT-Bench; Hands-On RAG for Production, ch. 6.

Four ways a judge leans

Known from research on LLM judges. None is fixed by a better prompt alone; each has a check you can run.

Bias	What the judge does	The check
Position	Prefers the first (or second) of two answers	Run both orders; keep verdicts that agree
Verbosity	Rewards longer, more polished answers	Grade claims against spans, not the whole reply
Self-preference	Favours answers from its own model family	A judge from another family; a human sample
Inconsistency	Gives the same answer different grades	Several runs; a reason that must quote a span

TIP

Sources: Zheng et al. (2023); Wang et al. (2023), LLMs are not fair evaluators; Panickssery et al. (2024), self-preference.

Position bias: the judge prefers one slot

The same two answers, shown in both orders. If the winner follows the slot, not the answer, the verdict is about position.

RUN 1: A FIRST

A	“Links expire after 30 minutes [H02].”	**WINS**
B	“Links last 30 minutes; request a new one after [H02].”	

Swap the order

RUN 2: B FIRST

B	“Links last 30 minutes; request a new one after [H02].”	**WINS**
A	“Links expire after 30 minutes [H02].”	

TIP The first slot won both times. Keep a verdict only when both orders agree; otherwise call it a tie.

Verbosity bias: longer answers score higher

Two answers with the same single fact. The padded one adds nothing a customer needs, and some judges grade it higher.

Short answer: judge gives 3 of 5

“Atlas reset links expire after 30 minutes [H02].”

One claim, supported

Long answer: judge gives 5 of 5

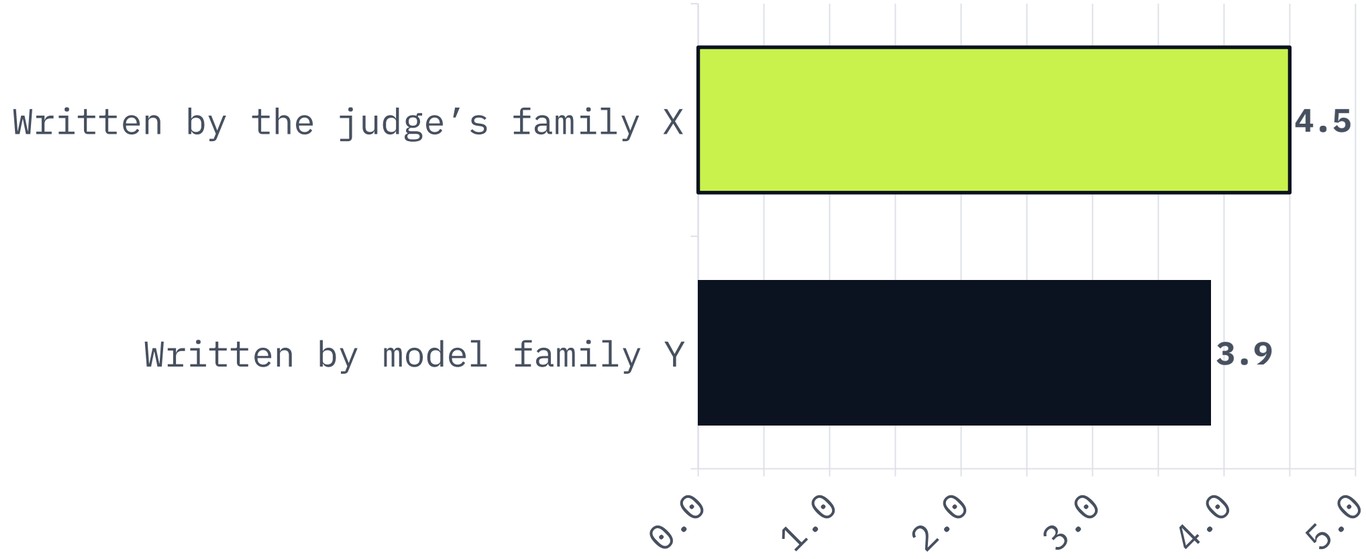
“Great question! Atlas takes security seriously. Reset links expire after 30 minutes, keeping your account safe [H02].”

Same fact, plus a claim no policy makes

TIP Grade claim by claim against the evidence: the padding then counts as an unsupported claim, not as quality.

Self-preference: a judge favours its own family

The same answer, written once by the judge’s model family and once by another. A judge can score its own style higher.



Matters most when the judge and the answering model are the same model, as in many quick setups.

Check: a judge from another family, and people on a sample.

Inconsistency: the same answer, another grade

Models sample their words, so a judge can change its verdict between runs, even at temperature 0.

RUN 1

F = 1

RUN 2

F = 0

RUN 3

F = 1

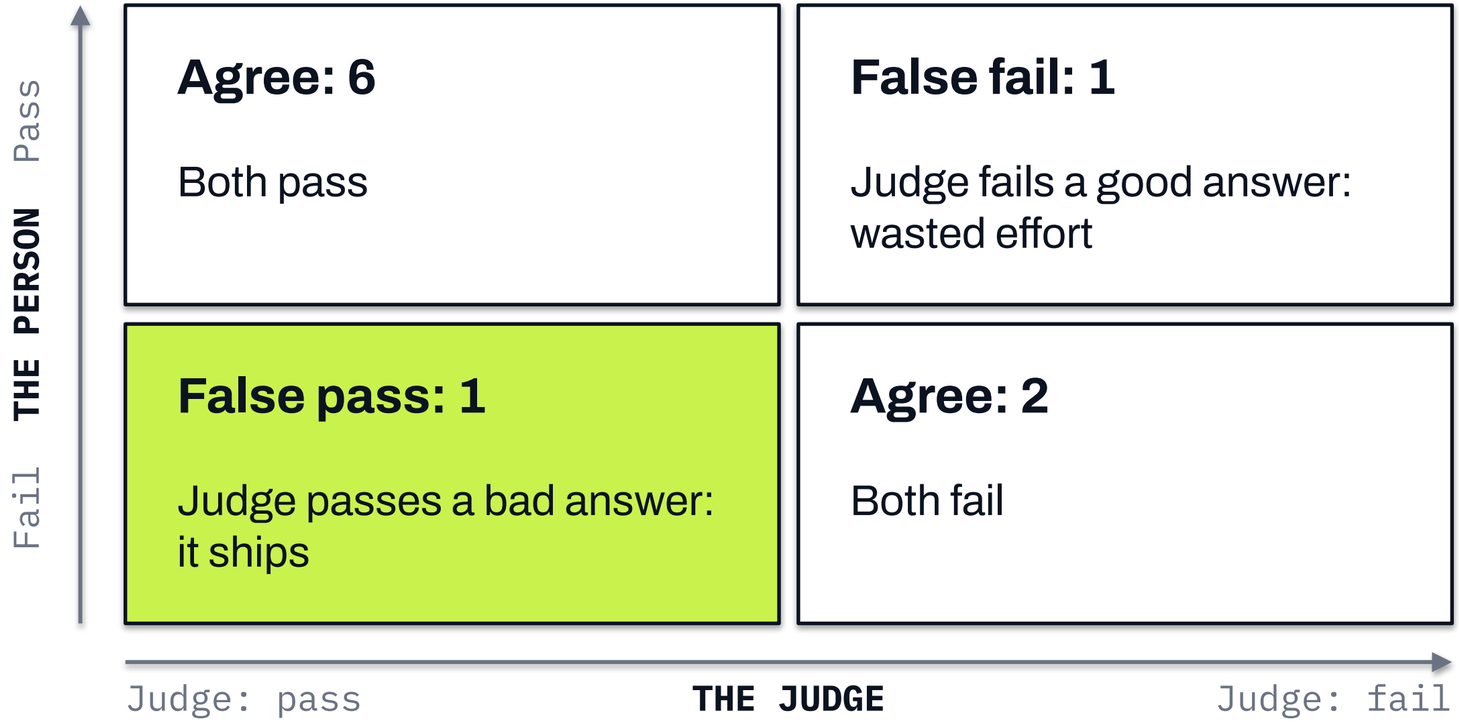
Same question, evidence, answer and rubric. Run the judge several times on a sample and count how often the verdict flips.

TIP Require a reason that quotes a span: a verdict you can check is worth more than a confident one.

MADE-UP EXAMPLE · 10 ANSWERS

Check the judge against people before you trust it

A person grades ten answers without seeing the judge's grades. Put the two side by side. They agree on 8 of 10.



Read the false pass first: it is the one that reaches a customer.

Ten answers is a sanity check, not proof: 8 of 10 could come from a judge that agrees far less often.

EXAMPLE · FROM LAB 07

Rules you can state exactly belong in code

The judge passed two answers taken from the wrong product's policy, each with a real quote. Four lines of code failed both.

QUESTION ABOUT BOREAL

Who can export all reports in Boreal?

ANSWER

Only workspace owners can export all reports in Boreal. [H03]

LLM judge	Product check in code
Finds the quote in H03 Faithful: pass	The question names Boreal H03 is an Atlas policy: fail

THE OVERVIEW

Evaluation tools at a glance

Each runs the checks from Parts 2 and 3 for you, many with an LLM judge inside. Read what a metric counts before you compare it with yours.

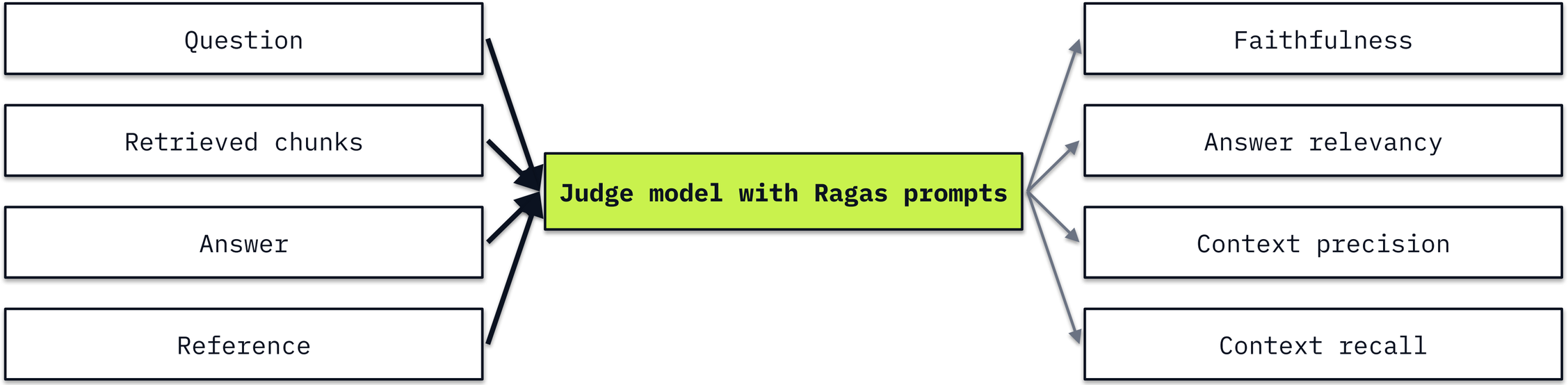
Tool	What it is	Typical use
Ragas	A library of RAG metrics and test-set generation	Score a golden set; generate questions
DeepEval	Evaluation written as tests, with thresholds	Fail a build when a score drops
TruLens	Records each run and scores the RAG triad	Inspect traces and feedback scores
MLflow	Tracks evaluation runs, traces and versions	Compare runs over time

WATCH The same name can mean different things: one tool's "context recall" counts labelled IDs, another's asks a judge. Others: Arize Phoenix, LangSmith.

TOOL 1 OF 4

Ragas: RAG metrics graded by a judge

You hand it questions, retrieved chunks, answers and (for some metrics) a reference. The paper (Es et al., 2023) proposed three of these checks; the library adds more.

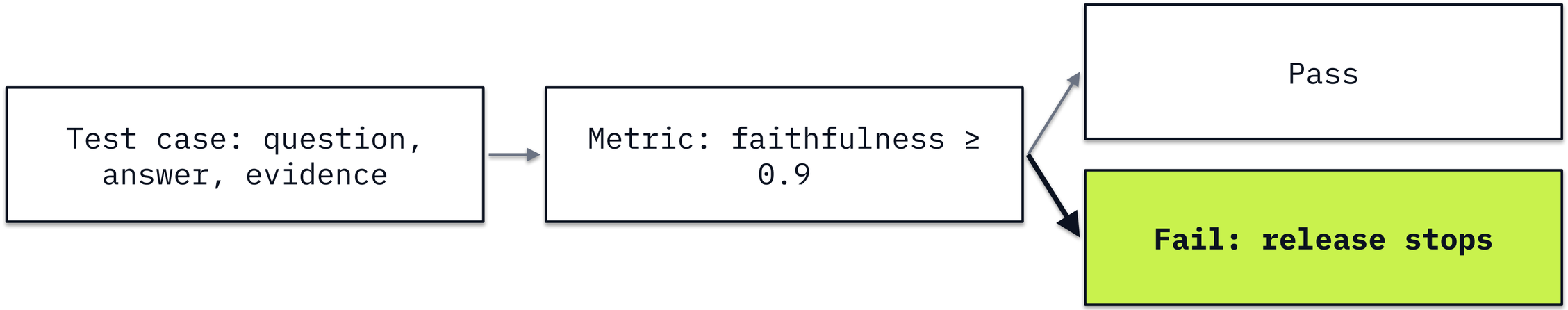


TIP It can also generate test questions from your documents (Part 4, label source 3).

TOOL 2 OF 4

DeepEval: evaluation written as tests

Each golden record becomes a test case with a metric and a threshold. A score under the threshold fails the test, like any unit test.

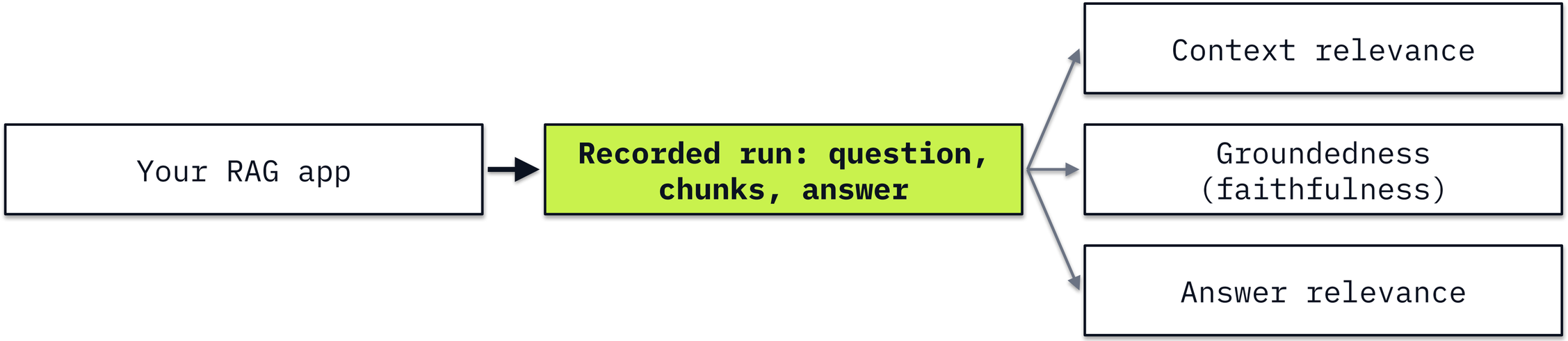


TIP Fits the release tests in Part 9: a change that lowers a score cannot ship.

TOOL 3 OF 4

TruLens: record each run, score the triad

It wraps your app, records every step of every request, and runs feedback functions on the record: the three checks from Part 3.

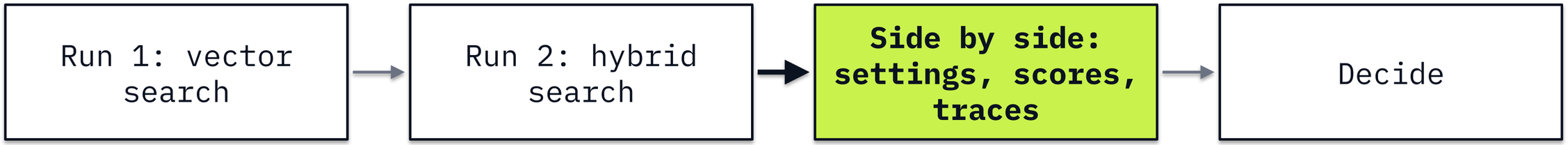


TIP Useful after release too: the same scores on live traffic.

TOOL 4 OF 4

MLflow: compare evaluation runs over time

Each evaluation is a run with its settings, scores and traces. Two runs side by side show what a change moved.



TIP The question-by-question table from Part 2, kept with the settings that produced it.

QUICK CHECK



The judge gives a high grade but quotes no words for the temporary-account claim. Do you accept the grade?

ANSWER

No: a grade without a quote cannot be checked

01

Read H01 and the claim yourself, and compare with a person’s label.

02

Ask the judge for a reason that quotes a span; reject replies without one.

LAB TIME

Lab 07: LLM judge

Label five answers yourself, then check an LLM judge against your labels before you trust it. In class: Parts B to E, without the two self-study parts of D. B’s labels are filled in: check each. Run every cell from the top.

In class

30 minutes

Folder 07_LLM_Judge_Calibration

Lunch

Next: better search

Keyword search, rank fusion and reranking, and how to tell whether they helped.

PART 6

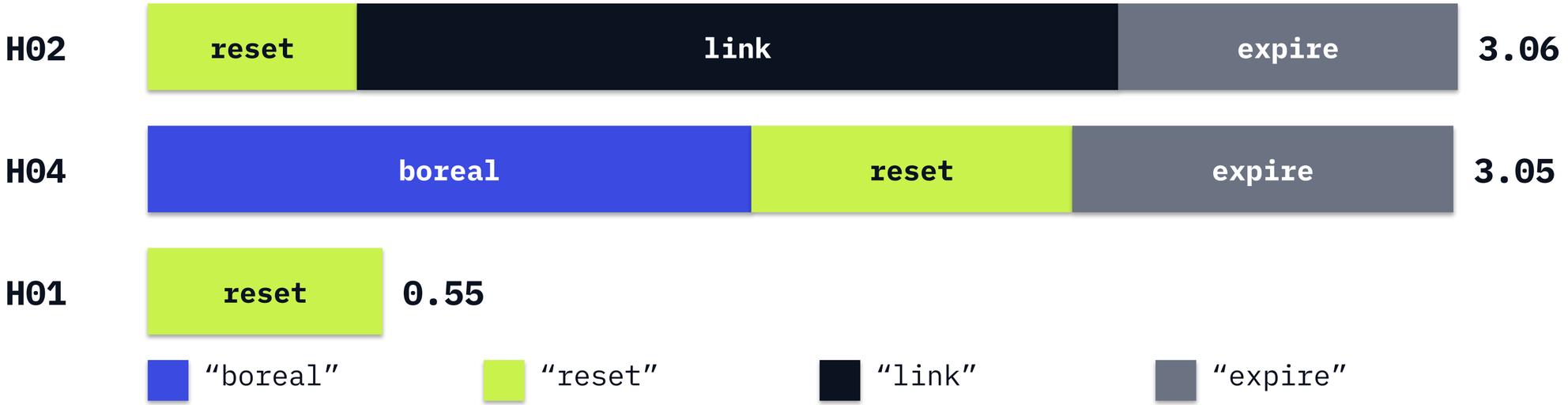
Better search: keywords, fusion, reranking

Day 1 named keyword search, hybrid search and reranking. Now see how each works on our policies, and how to tell whether it helped.

KEYWORD SEARCH · WORKED ON OUR POLICIES

BM25 scores a chunk by the query words it contains

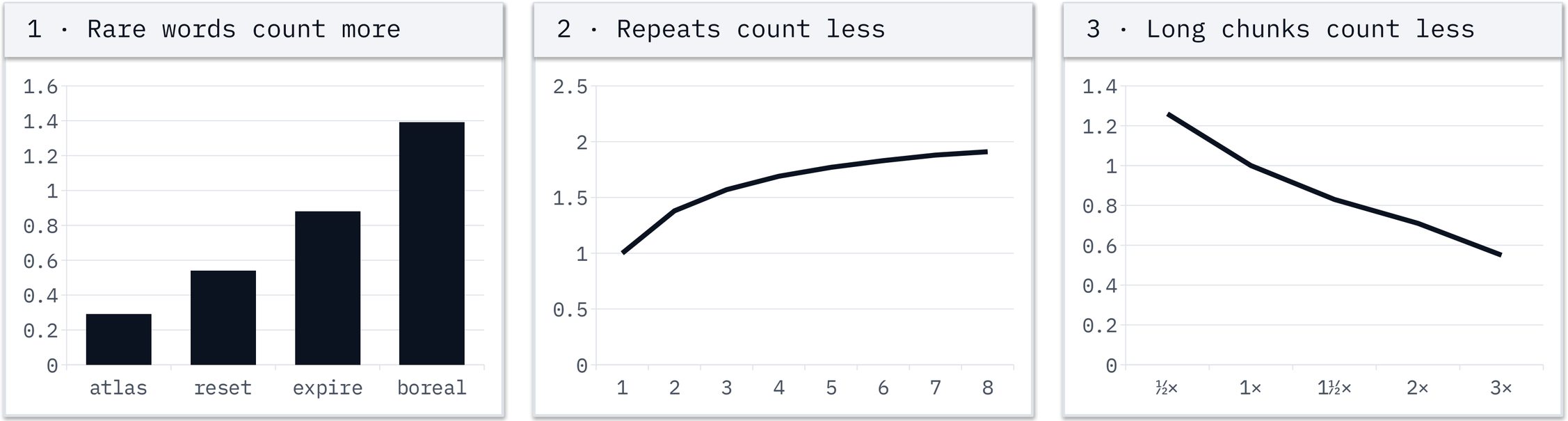
Query: “Boreal reset link expire”. Each word a policy contains adds to its score; rare words add more. Computed on H01 to H05.



WATCH H02, an Atlas policy, edges out H04 on a Boreal question: H02 says “link”, H04 says “links”. Real engines stem words to avoid this.

Three ideas inside the BM25 score

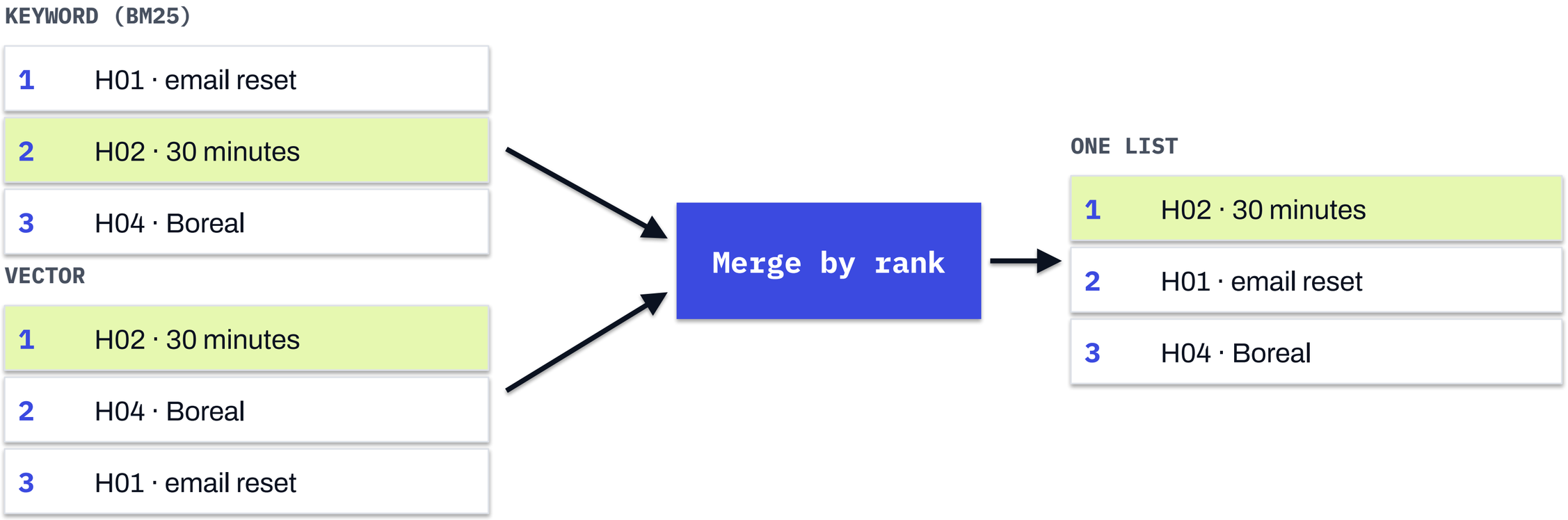
Rare words count more; each repeat of a word counts less than the last; a match in a long chunk counts less. Robertson and Zaragoza (2009).



Panel 1: the word's weight across our five policies. Panel 2: score as a word repeats 1 to 8 times. Panel 3: one match, as the chunk gets longer than average.

Hybrid search runs both searches, then merges

Query: “How long does the password recovery email stay valid?” Keyword search and vector search rank the policies differently.



Two ways to merge two rankings

Keyword scores and cosine scores are on different scales, so you cannot just add them. Merge by rank, or rescale the scores first.

Method	Uses	Needs	Good at
Reciprocal rank fusion (RRF)	Each chunk’s rank in each list	Nothing to tune (a constant, 60)	A safe default
Weighted score fusion	Each score, rescaled to 0–1	A weight, tuned on labelled questions	Can beat RRF once tuned

TIP

Sources: Cormack, Clarke and Büttcher (2009), RRF; Bruch et al. (2023), an analysis of fusion functions for hybrid retrieval.

FUSION METHOD 1 OF 2

Rank fusion adds 1 / (60 + rank) from each list

A chunk high in both lists collects two large terms; a chunk in one list collects one. Raw scores are never added.

$$\mathrm{RRF}(d) = \sum_{\text{lists}} \frac{1}{60 + \mathrm{rank}(d)}$$

Chunk	Keyword rank	Vector rank	RRF score	Merged rank
H02	2	1	**1/62 + 1/61 = 0.0325**	**1**
H01	1	3	1/61 + 1/63 = 0.0323	2
H04	3	2	1/63 + 1/62 = 0.0320	3

TIP Same lists as the previous page. The 60 keeps the top rank from swamping the rest; it is not k, the number of results.

WORKED EXAMPLE · MADE-UP LISTS

Rank fusion by hand: A, B and C

Keyword search returns B, A. Vector search returns A, C, B. Which comes first after fusion?

Chunk	Keyword rank	Vector rank	Sum	RRF
A	2	1	1/62 + 1/61	**0.0325**
B	1	3	1/61 + 1/63	0.0323
C	not in list	2	0 + 1/62	0.0161

A wins: it is near the top of both lists. B was first in one list but third in the other. C, found by one search only, gets one term.

WATCH Fusion rewards agreement between the two searches. It cannot tell whether A answers the question.

FUSION METHOD 2 OF 2 · MADE-UP SCORES

Weighted fusion rescales scores, then mixes them

Rescale each search’s scores so the best is 1 and the worst 0, then mix with a weight. Here the weight is 0.5 each.

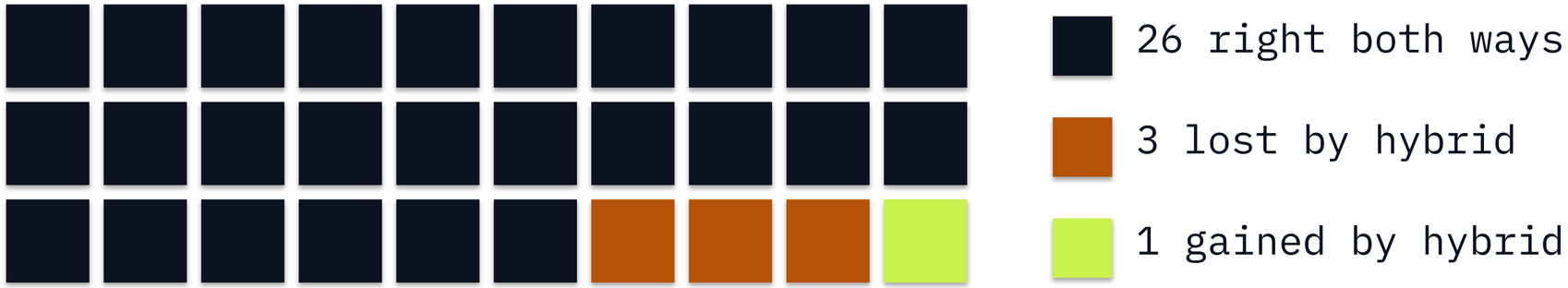
Chunk	BM25 → rescaled	Cosine → rescaled	0.5 × each, added
H01	3.1 → 1.00	0.58 → 0.00	0.50
H02	2.5 → 0.74	0.71 → 1.00	**0.87**
H04	0.8 → 0.00	0.62 → 0.31	0.15

H02 wins narrowly. Move the weight towards keyword search and H01 wins. Bruch et al. (2023) found a tuned weight can beat RRF; tuning needs a small set of labelled questions.

EXAMPLE · FROM LAB 08

Hybrid search did not help this collection

Vector search against hybrid search on 30 development questions, right chunk at rank 1. Each square is one question.



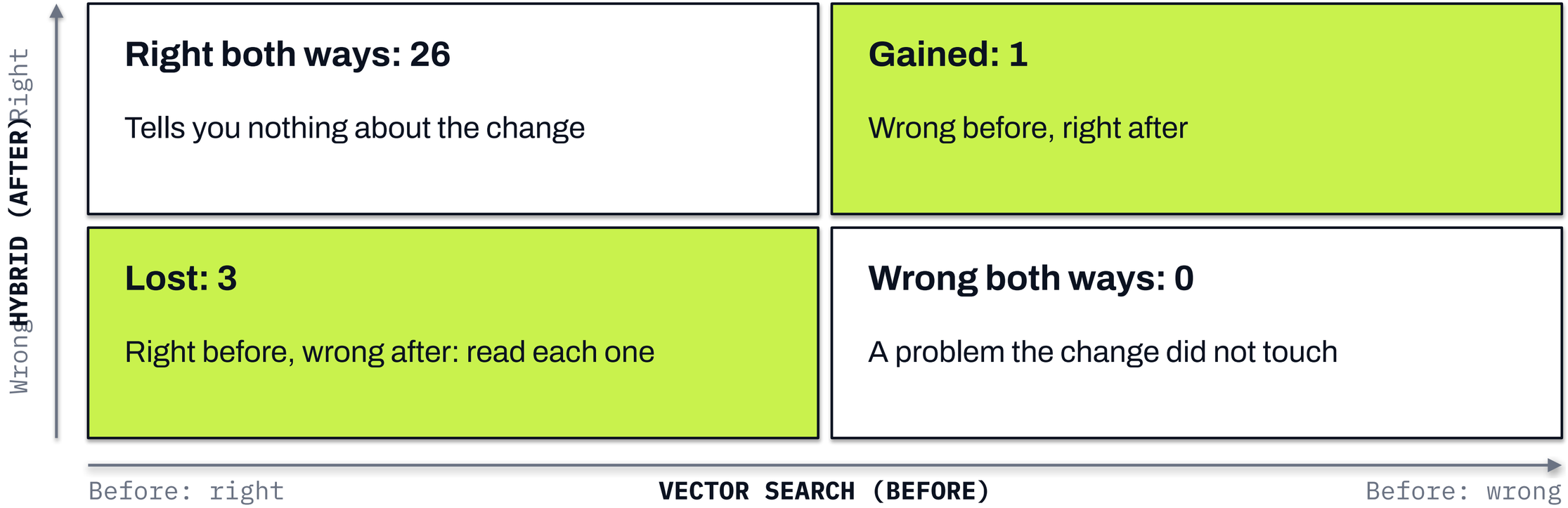
Vector search: 29 of 30 at rank 1. Hybrid: 27. Hybrid search is a common design, and on these questions it lost more than it gained.

TIP Labs 05 to 11 use the models in models.env (gpt-4o-mini, text-embedding-3-small in the saved runs). Another model can move these squares.

READING A COMPARISON

Count the questions that moved, not the average

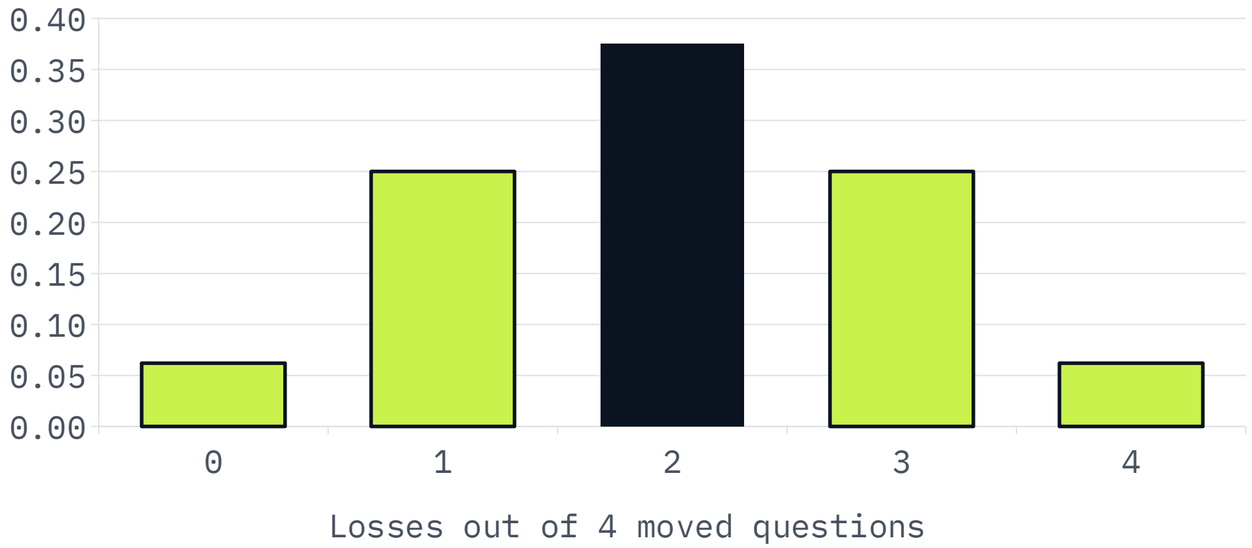
Put every question in one of four boxes. Only the two boxes where the result changed say anything about the change.



IS IT CHANCE?

A sign test asks whether the moves could be chance

If the change did nothing, each moved question is a coin flip: gain or loss. How surprising is 1 gain and 3 losses out of 4?



P-VALUE

10 / 16 = 0.63

outcomes at least as lopsided (lime)

A p-value is the chance of a result at least this lopsided if the change did nothing. 0.63 is common: chance explains it.

TIP With so few moved questions, no test can show a difference. Add questions where the change is meant to help.

QUICK CHECK



Hybrid search is in every vector database and many tutorials. Should we ship it here?

ANSWER

Not on this evidence

01

On these questions it lost three and gained one, and the split could be chance.

02

Keep vector search as the baseline and test the next change, a reranker, on the same questions.

RERANKING

Two stages: a fast search, then a careful model

A fast search narrows thousands of chunks to about 20. A slower, more accurate model reads those 20 and picks the best few.

All chunks: thousands

too many for a careful model

Fast search: top 20

embeddings or BM25, milliseconds

Reranker: best 3

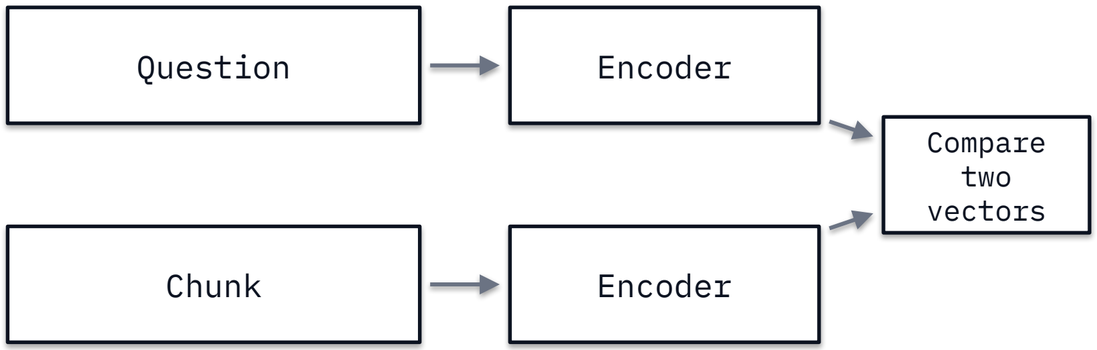
reads each with the question

Source: Hands-On RAG for Production, ch. 3 (the two-stage retrieval pipeline).

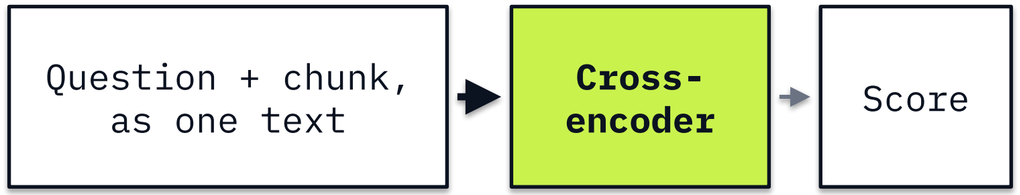
Read apart, or read together

Day 1’s embeddings read the question and each chunk apart, then compare two vectors. A cross-encoder reads the pair as one text.

BI-ENCODER (DAY 1): FAST, CHUNKS ENCODED ONCE



CROSS-ENCODER: SLOWER, READS BOTH AT ONCE



TIP A cross-encoder runs once per question–chunk pair at question time, so it only sees the short list.

THE OVERVIEW

Five kinds of reranker

All reorder a short list of candidates. They differ in how they read the text, what they cost and where they run.

Reranker	How it scores	Runs	Examples
Cross-encoder	Reads question and chunk together	Your machine or a GPU	bge-reranker, MS MARCO MiniLM
LLM reranker	A chat model orders the list	An API call	Any chat model (Lab 08)
Hosted rerank API	A service scores your candidates	Vendor’s servers	Cohere, Voyage, Jina
Late interaction	Matches word by word	Your machine; larger index	ColBERT
MMR (diversity)	Penalises near-duplicates	Plain arithmetic	Most vector stores

RERANKER 1 OF 5 · MADE-UP SCORES

A cross-encoder rescores each pair, then reorders

Question: “How long does an Atlas reset link stay valid?” Vector search put the answer third. The cross-encoder reads each pair.

VECTOR SEARCH (COSINE)

1	H01 · reset by email	0.62
2	H04 · Boreal, 10 minutes	0.58
3	H02 · Atlas, 30 minutes	0.55

Cross-encoder reads each pair →

AFTER RERANKING

1	H02 · Atlas, 30 minutes	9.1
2	H04 · Boreal, 10 minutes	2.3
3	H01 · reset by email	1.2

TIP Its scores rank candidates for one question; they are not probabilities. Nogueira and Cho (2019) reranked passages this way with BERT.

An LLM reranker returns the order in one call

Send the question and the numbered candidates; ask for their IDs, best first. Then check the reply names exactly the candidates.

THE REQUEST, IN WORDS

Order these by how well they answer: “How long does an Atlas reset link stay valid?”
[H01] reset by email…
[H04] Boreal links…
[H02] Atlas links expire…

THE REPLY

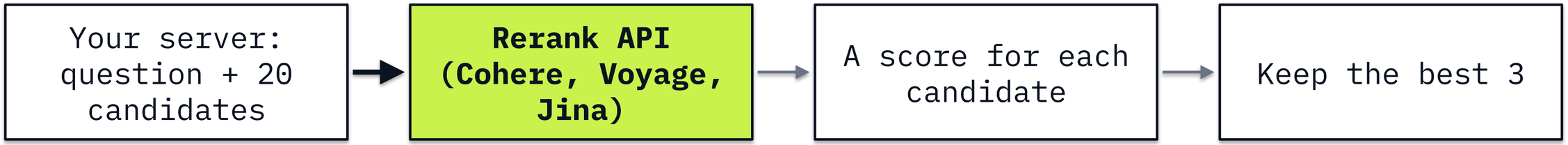
H02, H01, H04

Check in code: same three IDs, no new ones, none missing. Sun et al. (2023) showed chat models can rank passages this way.

WATCH Costs a model call per question, and the order of the candidates in the prompt can sway the reply.

A hosted rerank API scores your candidates

Send the question and your candidates to a rerank endpoint; get a relevance score for each. No model to host.



WATCH Your documents' text leaves your network on every question: check what your data rules allow.

Late interaction matches every word with every word

ColBERT keeps one vector per word. Each question word finds its best match in the chunk; the best matches are added up.

	atlas	reset	links	expire	30	minutes
atlas	**0.95**	0.30	0.20	0.10	0.05	0.05
link	0.20	0.40	**0.90**	0.35	0.10	0.15
valid	0.05	0.10	0.25	**0.70**	0.30	0.45

CHUNK SCORE

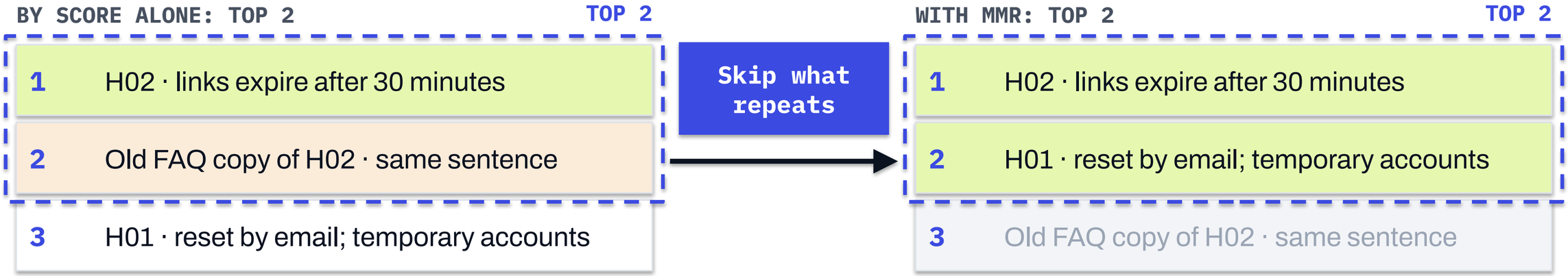
2.55

0.95 + 0.90 + 0.70, the lime cells

Closer reading than one vector per chunk, faster than a cross-encoder: word vectors are stored ahead of time. The cost is a larger index. Khattab and Zaharia (2020).

MMR prefers a chunk that adds something new

Maximal marginal relevance takes the best chunk, then the next one that is relevant and unlike those already chosen. Carbonell and Goldstein (1998).



TIP A weight sets the balance: 1 is pure relevance, 0 is pure novelty. Tune it on development questions.

QUICK CHECK



Search returned H01 and H04, but not H02. Can a reranker fix the answer?

ANSWER

No: a reranker only reorders what search returned

01

H02 is not in the list, so no reordering brings it back.

02

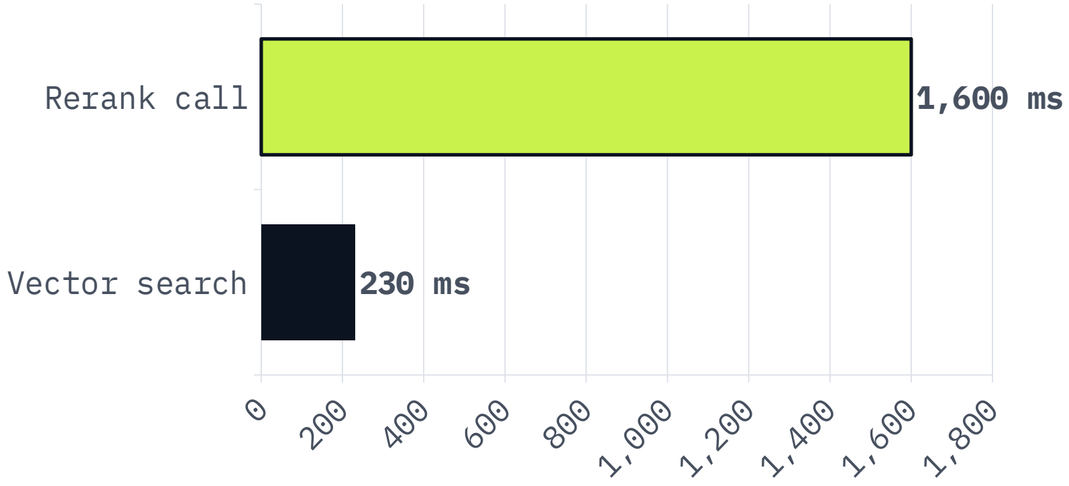
Search deeper (more candidates), or fix search first, then measure again on the same questions.

EXAMPLE · FROM LAB 08

A reranker can rescue a question, at a cost

A customer question where all three searches ranked the answering policy too low. One rerank call put it first.

Search	Where H02 ranked
Vector search	3rd
Keyword search (BM25)	6th
Hybrid search	4th
After one rerank call	**1st**



WATCH

Over 30 questions (Lab 08, self-study) the rerank call gained one question at rank 1 and lost one. Better on some questions is not better overall.

LAB TIME

Lab 08: Hybrid search and reranking

Add keyword search and rank fusion to vector search and decide question by question whether it helps; then one rerank call. In class: Parts B to D, then Part E up to Checkpoint E (one rerank call).

In class

40 minutes

Folder 08_Hybrid_Search_And_Reranking

Break

Next: follow-up questions

Questions that name nothing, and how to rewrite them before search.

PART 7

Follow-up questions and query rewriting

In a conversation the second question often names nothing. Search reads only the words it is given.

EXAMPLE · FROM LAB 09

Searched as typed, a follow-up finds the wrong product

The user asked about Boreal, then a follow-up that names no product. Search sees only the follow-up, and finds Atlas.

USER
How do I reset my Boreal password?

ASSISTANT
Boreal password reset uses a mobile app [H04].

USER
How long is the link valid?

SEARCH SEES ONLY THE LAST TURN

1 H02 · Atlas: 30 minutes

2 H04 · Boreal: 10 minutes

THE ANSWER

“30 minutes [H02]”

quote found in H02, and the wrong product

DEFINITION

Standalone query: the follow-up rewritten so it makes sense on its own, using the conversation, without answering it.

“How long is the link valid?” after a Boreal question becomes “How long is a Boreal password reset link valid?”

Five ways to rewrite a query

All change what search is given, not what the user asked. Each adds a model call before search.

Method	What search is given	Helps when
Standalone rewrite	The follow-up, made complete	A follow-up names nothing
Multi-query	Several phrasings, then merged	Recall is low
HyDE	A pretend answer	Questions look unlike documents
Decomposition	One sub-question per part	A question asks two things
Step-back	A broader question first	A general rule decides it

TIP

Sources: Building AI Agents with LLMs, RAG and Knowledge Graphs, ch. 6; Unlocking Data with Generative AI and RAG, ch. 14.

REWRITE METHOD 1 OF 5

A standalone rewrite fills in what is missing

One model call reads the conversation and rewrites the last turn. Same search, different words, different first result.

SEARCH: “HOW LONG IS THE LINK VALID?”

1	H02 · Atlas: 30 minutes
2	H04 · Boreal: 10 minutes

Rewrite with the history



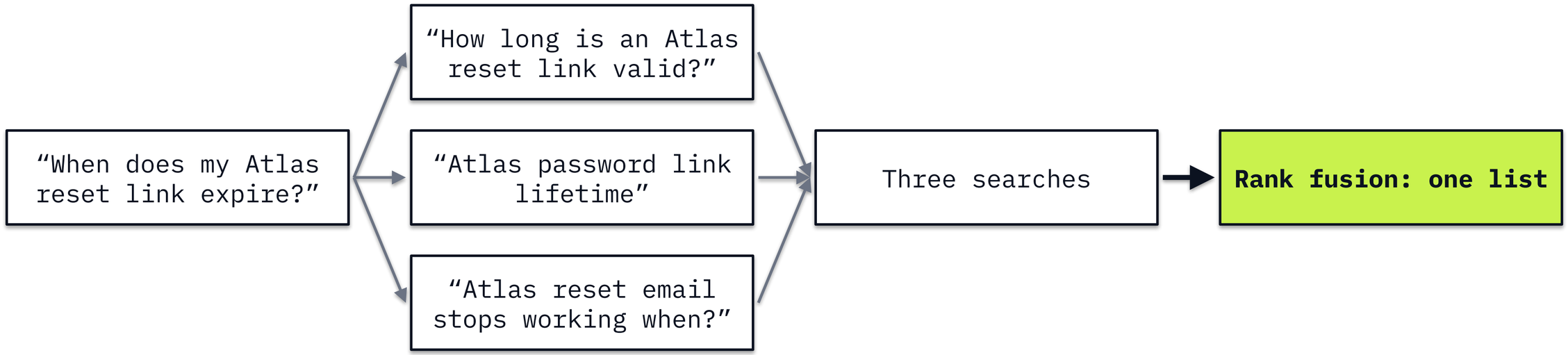
SEARCH: “BOREAL RESET LINK VALIDITY”

1	H04 · Boreal: 10 minutes
2	H02 · Atlas: 30 minutes

WATCH The rewrite asks; it never answers. It must not copy an earlier answer’s number into the query.

Multi-query: several phrasings, then merge

A model writes a few versions of the question; each is searched; rank fusion (Part 6) merges the lists. Also called RAG-Fusion.

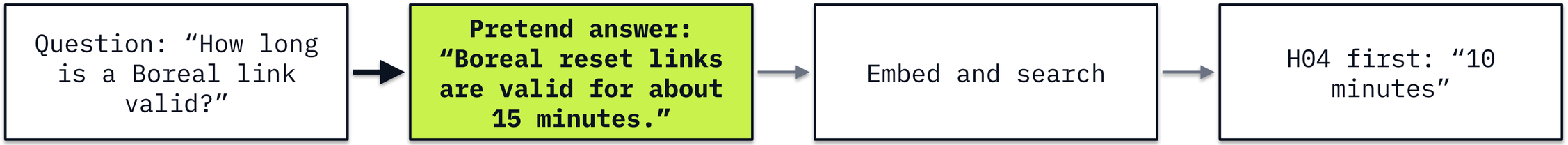


TIP More recall, at more searches and one extra model call per question.

REWRITE METHOD 3 OF 5

HyDE: search with a pretend answer

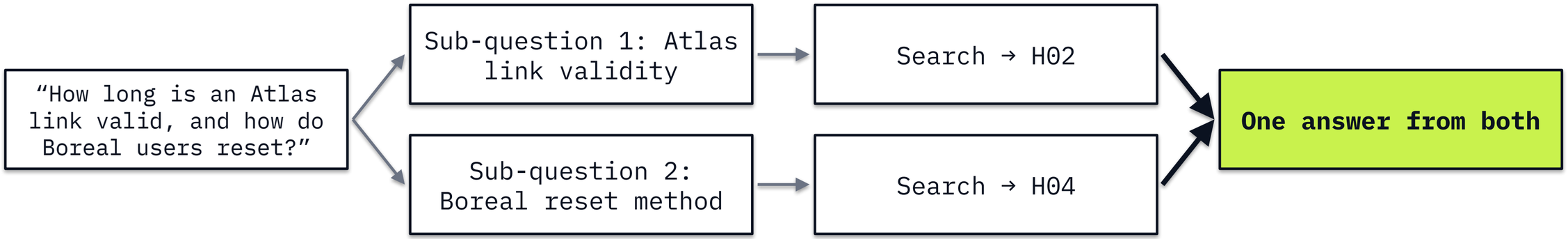
HyDE (hypothetical document embeddings): a model writes what an answer might look like; that text is embedded and searched. Answers look more like documents than questions do. Gao et al. (2023).



WATCH The pretend answer’s 15 minutes is wrong, and it does not matter: it is only used to search, never shown or cited.

Decomposition: split a two-part question

A question that asks two things gets two searches. One search for both often finds chunks for one half only.

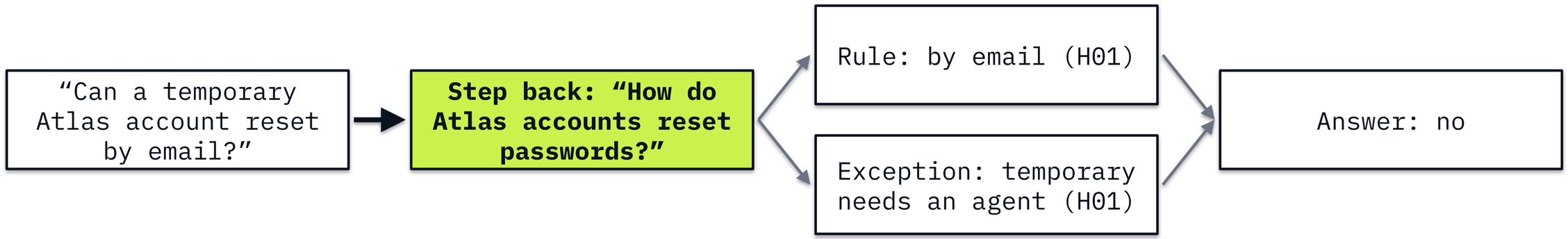


TIP Source: Unlocking Data with Generative AI and RAG, ch. 14 (query decomposition).

REWRITE METHOD 5 OF 5

Step-back: ask the broader question first

Some questions depend on a general rule. Search for the rule first, then for the specific case. Zheng et al. (2023), Take a step back.



TIP Useful when the specific words rarely appear in the documents but the rule does.

Check every rewrite in code before you search

A rewrite is a model reply, so check it like one. Two rules catch the common failures: the product kept, no new facts.

Rewrite	Names the product?	Adds a number?	Verdict
Boreal password reset link validity	Yes: Boreal	No	**Search with it**
How long is the link valid?	No	No	Fail: product lost
Is the Boreal link valid for 30 minutes?	Yes	Yes: 30, from the Atlas answer	Fail: leaked fact

TIP Both checks are a few lines of code. A rule you can state exactly does not need a model.

History says what the user means; documents say the facts

The conversation helps interpret the question. Only retrieved evidence supports the answer. An earlier answer is never evidence.

Conversation history: meaning

Which product is “it”?
Which account type?
What does the user want now?

Retrieved evidence: facts

Search again for this turn
Cite what is in the evidence
Recheck permissions every turn

TIP When the history does not settle it (“How long does it last?” after Atlas and Boreal), ask which one.

QUICK CHECK



Atlas and Boreal both came up earlier. The user asks “How long does it last?” What should the system do?

ANSWER

Ask which product; do not guess

01

A guessed product gives a confident, cited answer to the wrong question.

02

Detect it in code: two products in the history, none in the question → ask.

LAB TIME

Lab 09: Follow-up questions

Search a follow-up as typed and watch it find the wrong policy; rewrite it with one chat call; check the rewrite in code. In class: Parts A to C, except the ten-run count at the end of Part A.

In class

25 minutes

Folder 09_Follow_Up_Questions

PART 8

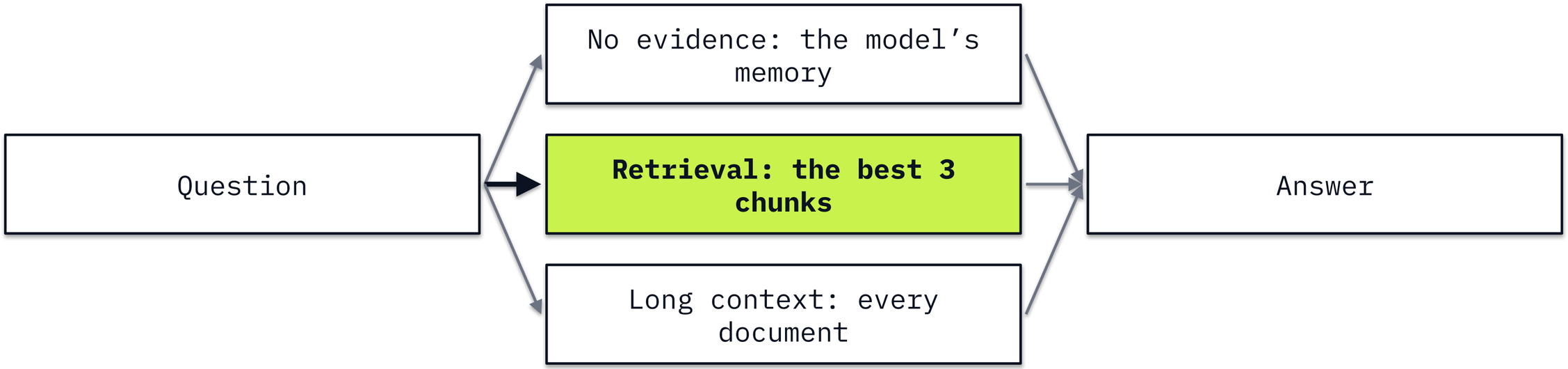
Long context or retrieval

Models read very long prompts now. Here is what sending every document costs, and what it loses.

THREE ROUTES

Three ways to answer the same question

No evidence, a few retrieved chunks, or every document in the prompt. Day 1 compared them in principle; Lab 10 measures them.

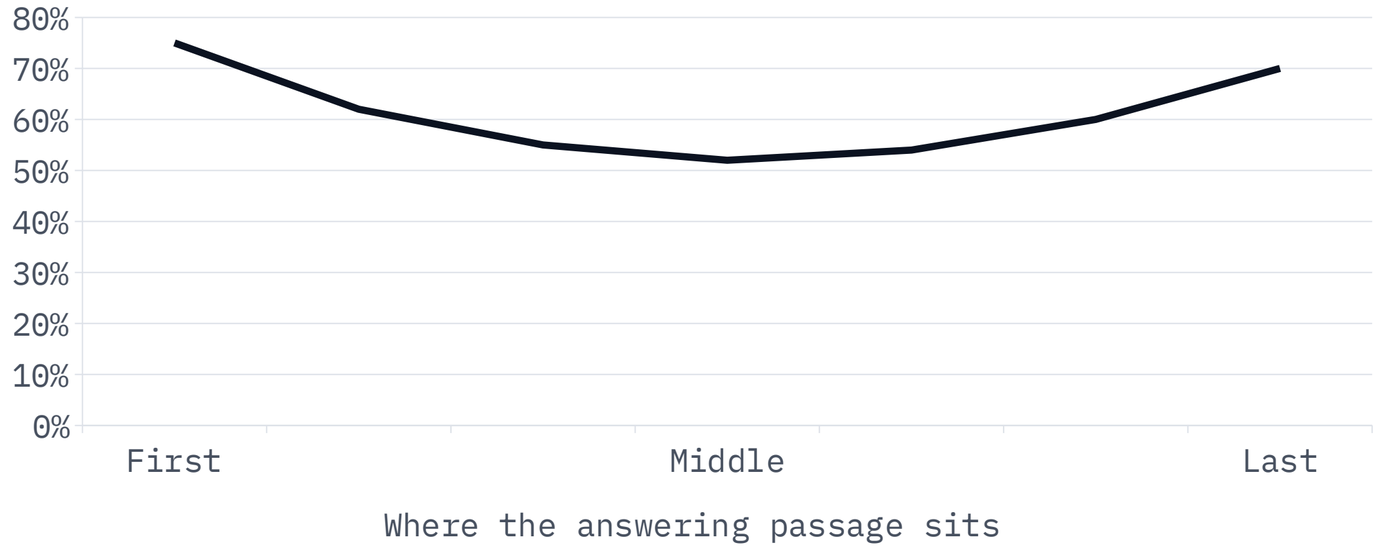


TIP Long context skips the index, and pays for every token of every document on every question.

WHAT RESEARCH FOUND · SHAPE ONLY

Models miss facts in the middle of a long prompt

Liu et al. (TACL 2024) moved the passage that answers a question through a long prompt. Accuracy was highest at the ends.



The curve shows the U shape the paper reports, not its numbers.

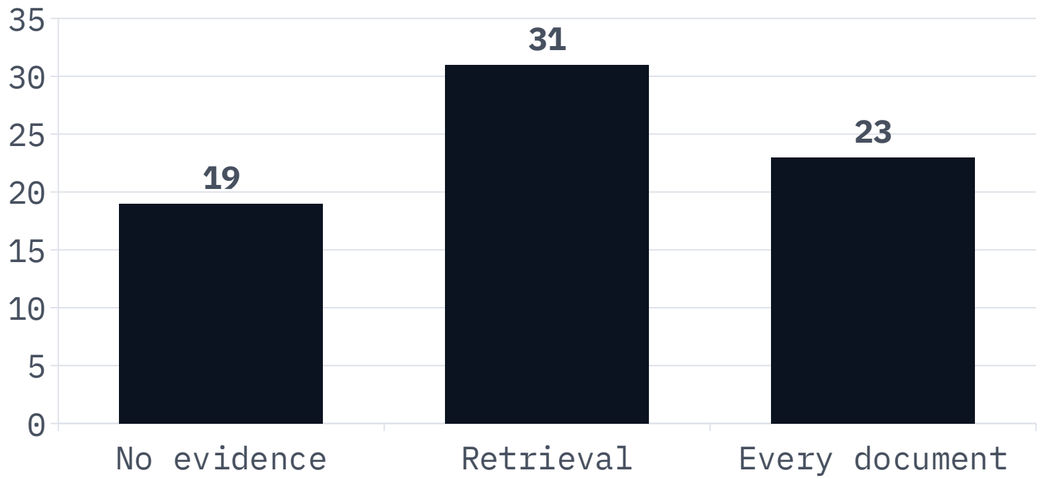
Retrieval keeps prompts short, so the passage that matters has nowhere to hide.

EXAMPLE · FROM LAB 10 (SELF-STUDY)

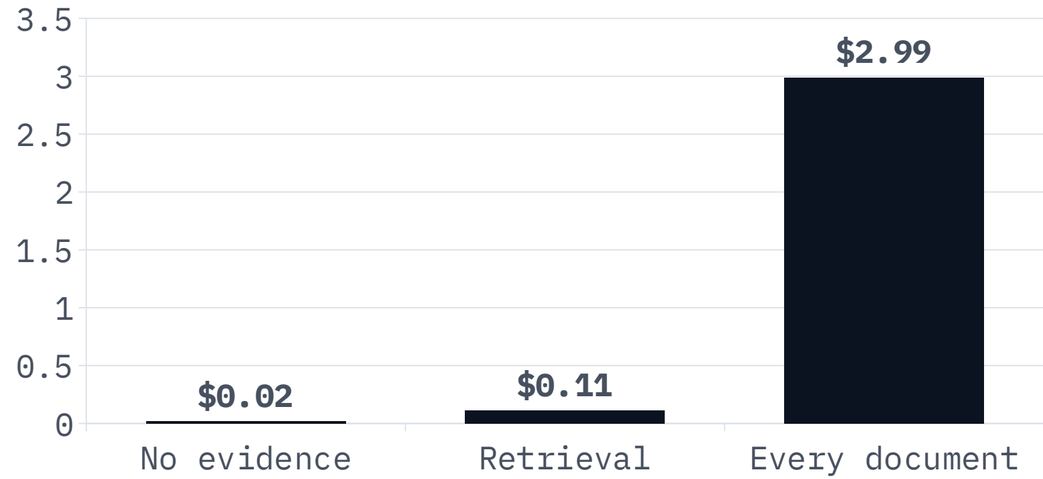
Everything fit in one prompt, and answered fewer

The same 34 questions three ways. The whole collection fit in the prompt, and still answered fewer right than retrieval.

Answers passing the answer check (of 34)



US dollars per 1,000 questions

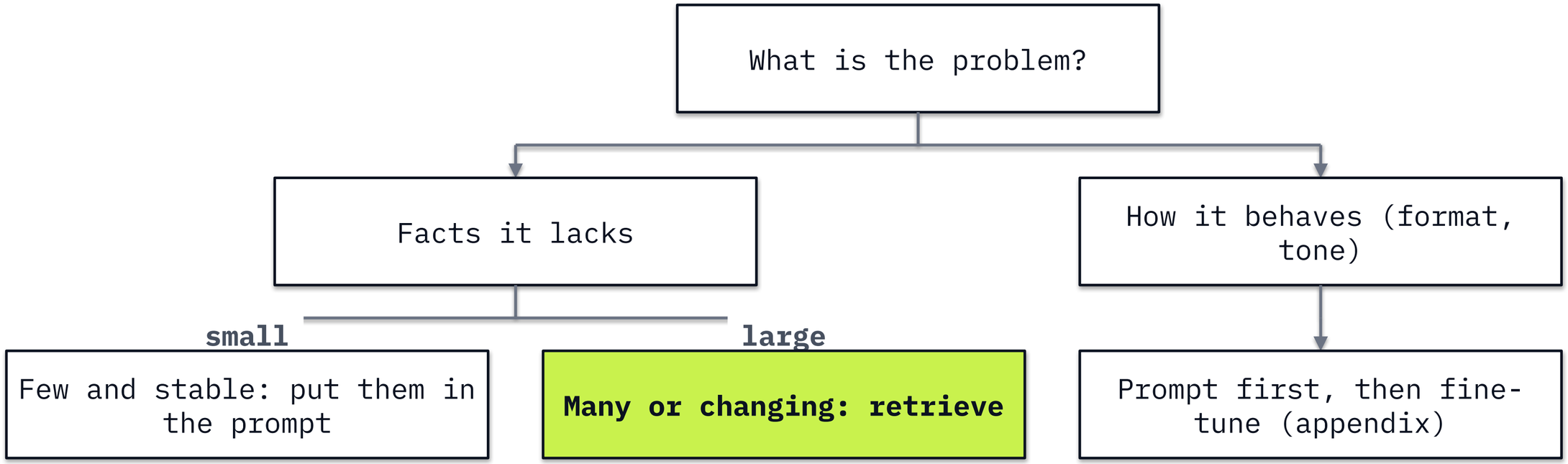


TIP

Given the right passage alone, the model answered the questions it missed in the full prompt.

Choose by what failed and how often facts change

Start from the problem, not the tool. Li et al. (EMNLP 2024) route each question: RAG first, long context when the model says the chunks cannot answer.



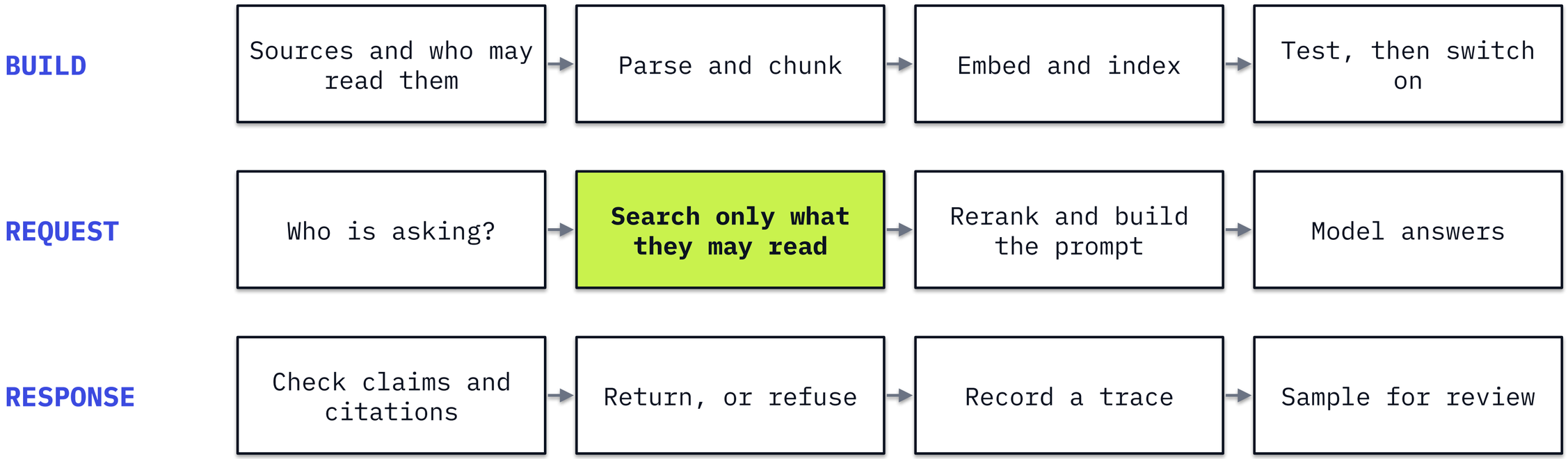
PART 9

From notebook to service

A service answers people you don’t know, from documents that change. Decide who may see what, test every change, keep watching.

A RAG service has three paths: build, request, response

Build the index ahead of time; answer each request; check every response before it leaves. Hands-On RAG for Production, ch. 4.



Five checks before and after release

Each has its own page next. The first three stop a bad release; the last two catch what gets through.

Check	The question it answers	When
Permissions	Can evidence hold text this user may not read?	Every request
Versioned index	Which documents made this answer? Can we go back?	Every update
Release tests	Does the change break a rule?	Before release
Monitoring	Are real answers getting worse?	After release
Cost and latency	What does each stage cost in money and time?	Always

TIP

Sources: Hands-On RAG for Production, ch. 4 and 6; OWASP Top 10 for LLM Applications 2025 (a security checklist), “vector and embedding weaknesses”.

CHECK 1 OF 5

Permissions: filter before the model sees any text

A prompt that asks the model to hide text cannot un-send it. Filter inside the search, so the text never reaches the prompt.

Filter inside the search

Search only chunks this user may read
Restricted text never reaches the prompt
The safe default

Filter after the search

Search everything, then drop what they may not read
The top 3 can all be dropped: no evidence left
One bug, and restricted text gets through

TIP Label every document with who may read it when you build the index, not at question time.

EXAMPLE · FROM LAB 11 (SELF-STUDY PARTS)

List what each user may read, not what to hide

A change filtered with a list of what to hide. One internal note nobody had listed reached customers' evidence.

Deny-list: hide what is listed

A note with no label slipped through

In the evidence for six customer questions

No reply repeated it: only a test on the evidence caught it

Allow-list: only what is listed

Customers see public documents only

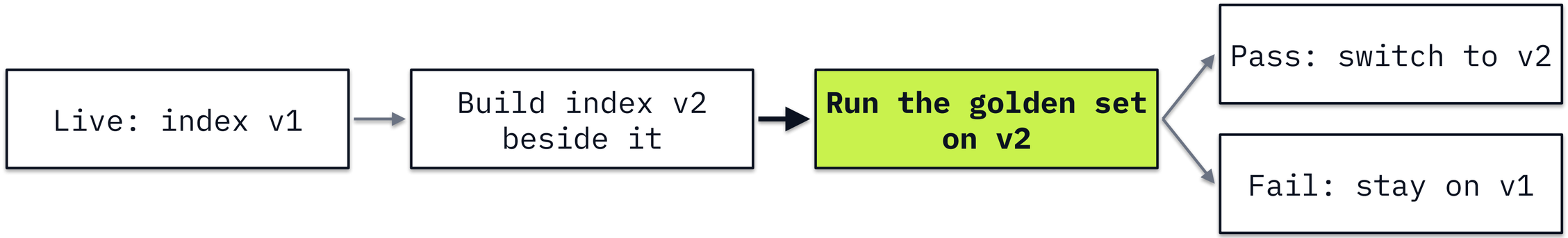
An unlabelled note is excluded by default

All five release tests pass

CHECK 2 OF 5

Build a new index beside the old, test, then switch

Never change the live index in place. Build the new version next to it, run the golden set on it, switch, and keep the old one for rollback.

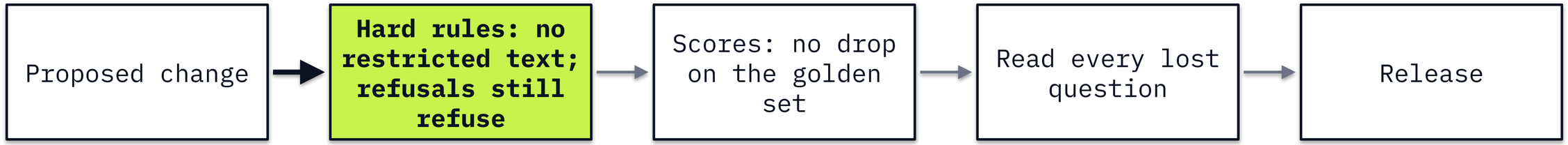


TIP Remove the old version of a changed policy by its ID. Never mix old vectors with changed text.

CHECK 3 OF 5

Release tests: a broken rule blocks the release

Hard rules are checked first, and any failure stops the release. Only then do averages and question-by-question changes count.

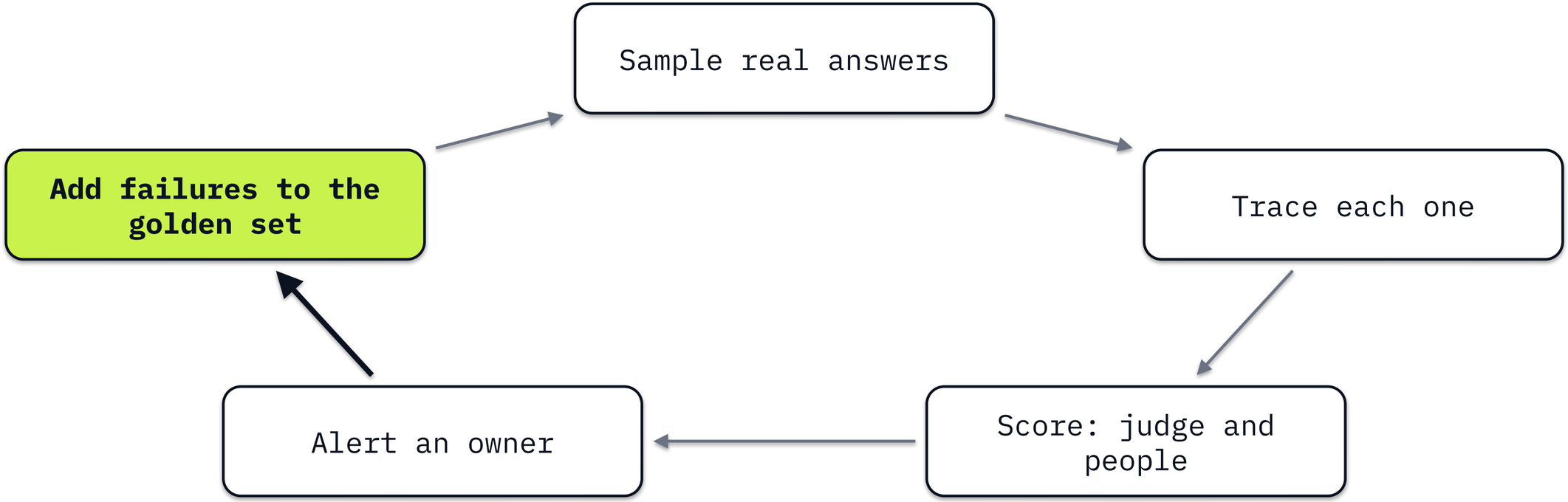


TIP Write them as ordinary tests that run on saved results, with no API key, on every change.

CHECK 4 OF 5

After release, sample real answers and watch

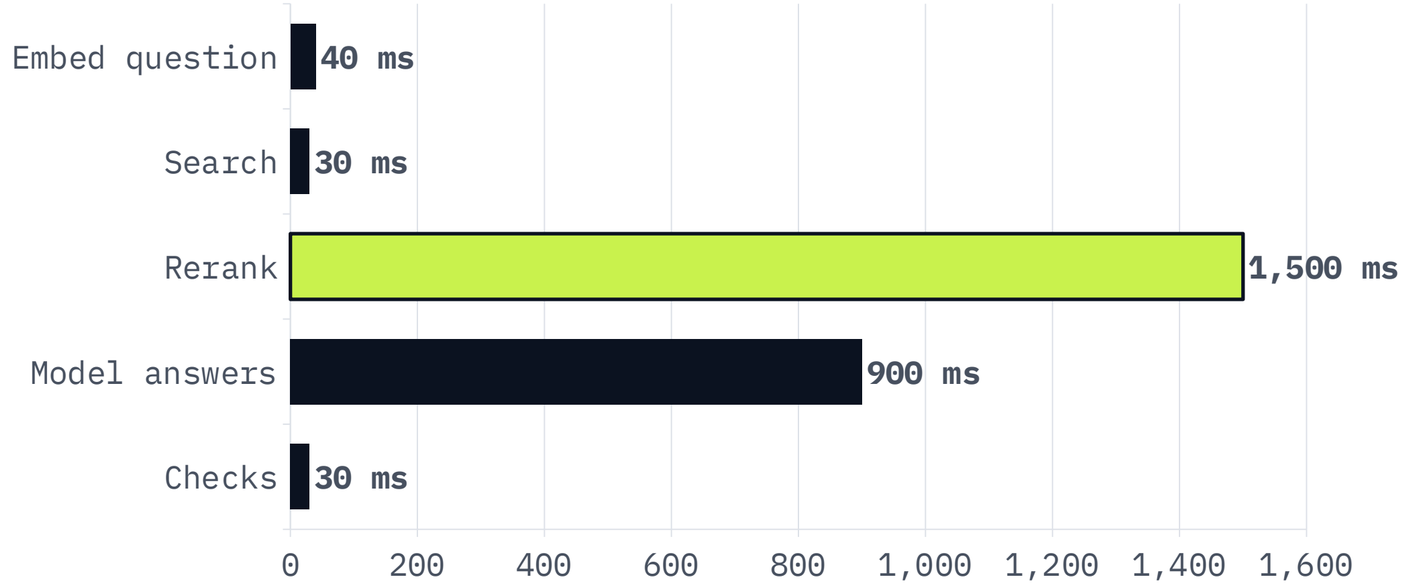
Live questions differ from your golden set. Review a sample every week, trace each answer, and alert an owner when a rate moves.



CHECK 5 OF 5 · MADE-UP TIMINGS

Cost and latency: measure each stage

Time each stage on your own hardware or API. Report p50 (half of requests are faster) and p95 (95 of 100 are faster).



In this made-up request the reranker is the slowest stage. Caching repeated prompts and choosing a smaller model are the usual levers; measure before choosing.

TIP Averages hide the slow tail: users feel p95, not the mean.



The average score went up, and one customer’s evidence held an internal note. Ship the change?

ANSWER

No: a broken access rule blocks the release

01

A better average cannot buy back a leak. The access test fails on its own.

02

Fix the filter, rerun every release test, then compare question by question.

LAB TIME

Lab 11: Release gate

Keep the documents a customer may not read out of their evidence, and check the filter by reading the replies, not only the hits. In class: Parts A and B.

In class

20 minutes

Folder 11_Release_Gate

PART 10

Where RAG goes next: agents and graphs

Two directions the field is taking, and when each is worth what it costs.

WHERE TODAY FITS

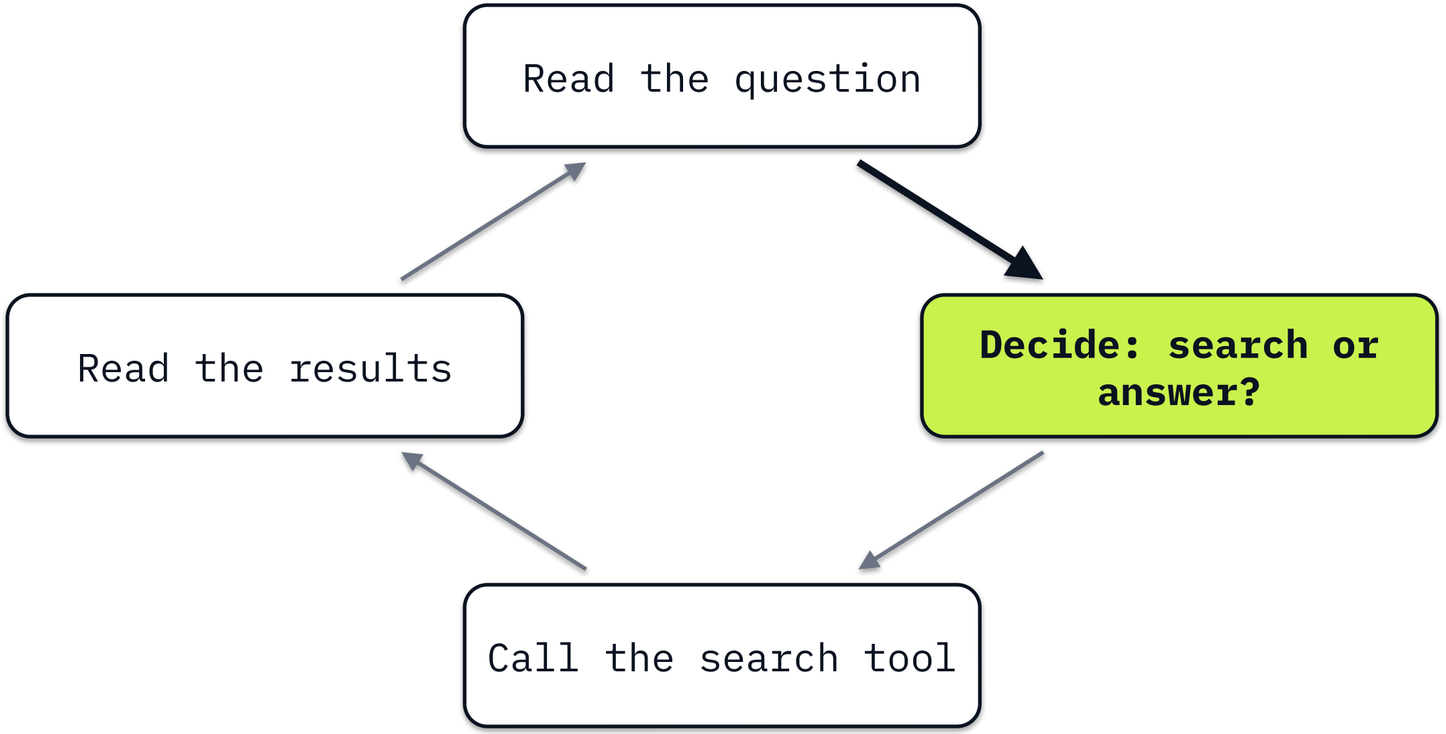
RAG comes in levels; today was advanced RAG

Surveys describe RAG growing in levels: naive, advanced and modular (Gao et al., 2023), then graph and agentic RAG (Singh et al., 2025).

Level	What it adds	In this course
Classic (naive) RAG	Search, put chunks in the prompt, answer	Day 1
Advanced RAG	**Hybrid search, reranking, query rewriting**	**Day 2: today**
Agentic RAG	The model decides when and what to search	Overview, next page; Bonus lab
GraphRAG	Searches a graph of things and their links	Overview, after that

Agentic RAG: the model decides when to search

Instead of one search with the question, the model gets search as a tool. It can search, read, search again, or answer without searching.



A stop rule in code caps the number of searches.

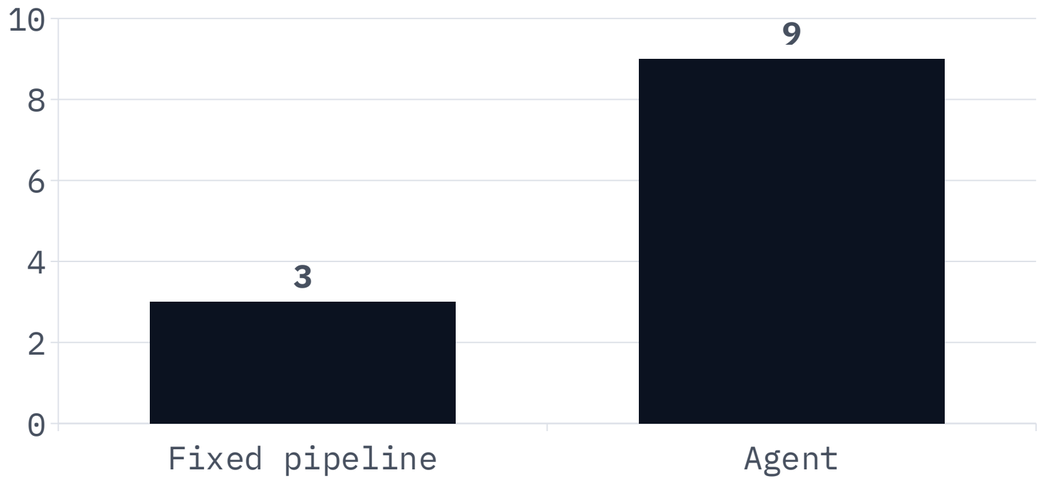
Sources: Hands-On RAG for Production, ch. 7; Asai et al. (2023), Self-RAG.

EXAMPLE · FROM THE BONUS LAB (SELF-STUDY)

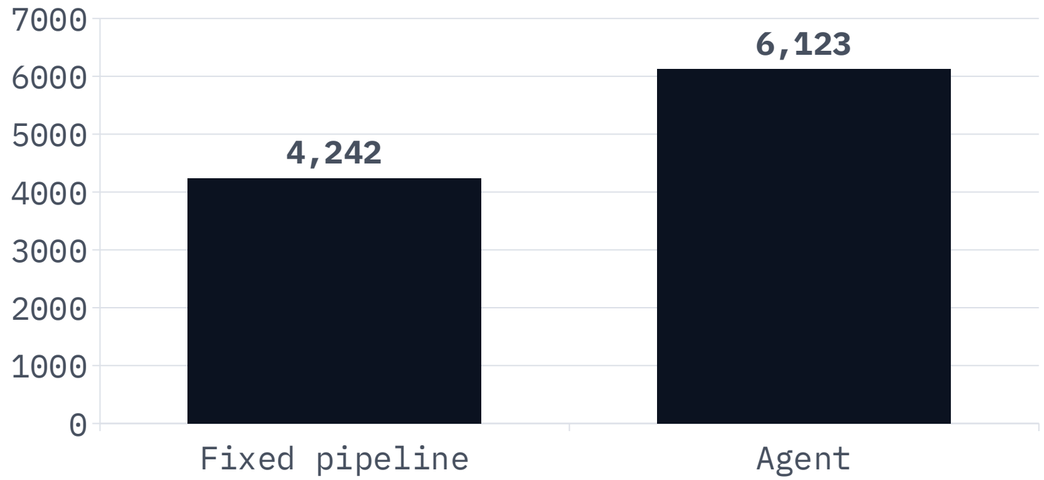
An agent can split a question the pipeline gets half wrong

A question with two parts. The fixed pipeline searched once and missed one half; the agent searched twice, once per part.

Two-part questions: right out of 9 tries



Input tokens for those questions

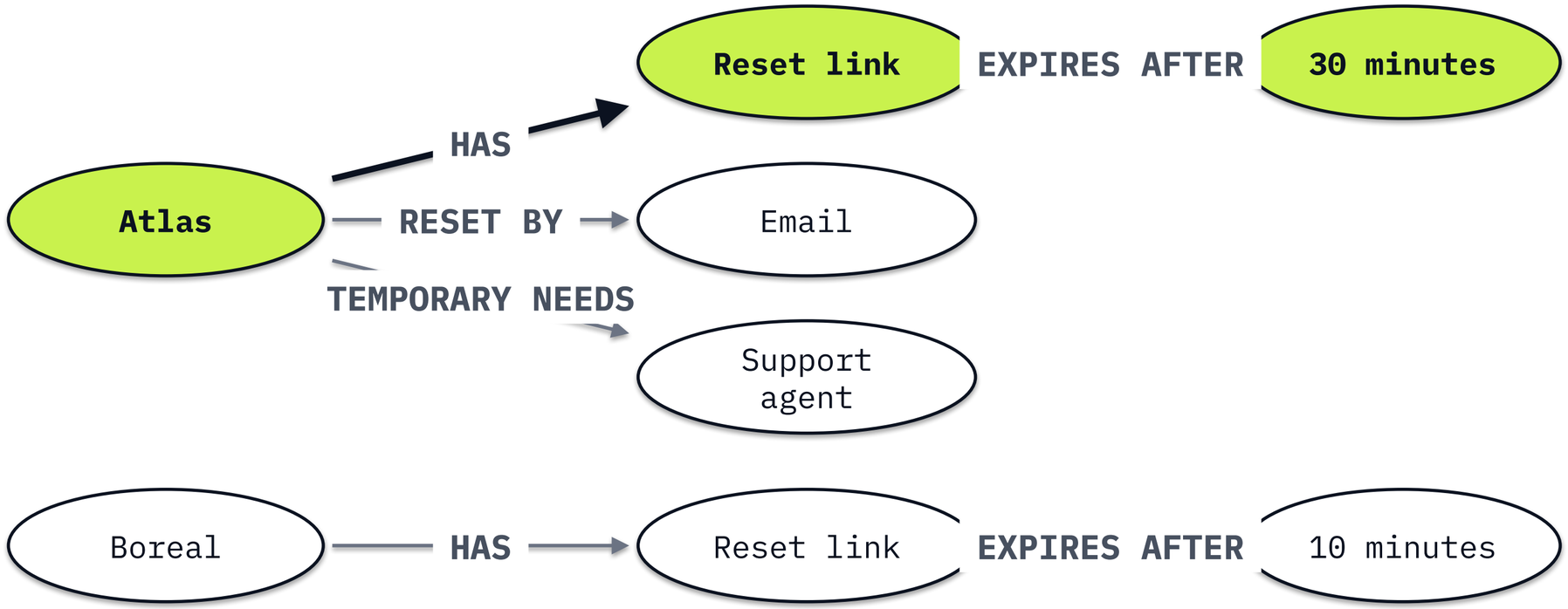


TIP

On a one-fact question both were right, and the agent used twice the model calls. Agents pay off on questions that need several lookups.

GraphRAG: search a graph of things and their links

A model reads the documents and extracts things (products, rules, roles) and how they connect. Questions then follow the links.



Each link remembers the chunk it came from, so answers can still cite.

GraphRAG helps with questions about everything

Ordinary RAG answers questions one chunk can answer. Some questions are about the whole collection. Edge et al. (2024).

A local question

“How long is an Atlas reset link valid?”
One chunk answers it: H02
Classic RAG is enough

A global question

“What are the main themes across all our policies?”
No single chunk answers it
GraphRAG summarises groups of linked things

WATCH The cost: many model calls to build the graph, again whenever documents change. Try classic RAG first.

CLOSE

What you can do now

- Measure search with hit@k, MRR and nDCG, and answers claim by claim
- Build a golden set, and check an LLM judge against people
- Improve search with fusion and reranking, and read the moved questions
- Rewrite follow-ups, and ship behind permissions and release tests

AFTER CLASS Lab 10, the Capstone, the Bonus, Optional A and B, and the final challenge: next page.

KEEP GOING

Labs to try after class

Each lab’s README says what it teaches, which labs it builds on, and what you should see.

Lab	What it teaches
Lab 10: Long context or retrieval	Measure both routes on answers, tokens and cost
Capstone: a RAG service	A web page with sign-in, traces and the release gate in CI
Bonus: agentic search	Let the model decide when and what to search
Optional A: LlamaIndex	The same pipeline, and the defaults it sets for you
Optional B: managed search	A hosted search service, and what it hides
Final challenge	Find the failing stage, change one thing, compare every record

APPENDIX

Fine-tuning, for reference

Not needed for today. When changing the model is the right fix, and what is changing on OpenAI’s platform.

APPENDIX

Fine-tune for behaviour, not for facts

Fine-tuning updates the model's weights from examples. It suits a repeated behaviour; a changed fact belongs in a document.

Good reasons to fine-tune	Not a fine-tuning problem
Always answer in one format	A policy changed: update the document
A tone or style every answer needs	Search missed: fix search
Refuse the same kind of question every time	Rare facts: retrieval did better in studies

TIP

Ovadia et al. (EMNLP 2024): retrieval beat unsupervised fine-tuning for new facts.
Soudani et al. (2024): clearest for less popular facts.

APPENDIX · WORKED EXAMPLE

LoRA trains a small add-on, not the whole model

LoRA freezes a weight matrix W and learns two thin matrices, B and A, whose product is added to it. Hu et al. (2021).

$$W_{\text{new}} = W + BA$$

Matrix	Shape	Numbers	Trained?
W (one weight matrix)	896 × 896	802,816	No: frozen
B and A (rank 8)	896 × 8 and 8 × 896	**14,336**	Yes

TIP 14,336 / 802,816 = 1.8% of one matrix, for a small open model (Qwen2.5-0.5B) whose matrices are 896 × 896. A full run adds this for every adapted matrix.

APPENDIX · CHECKED 27 OCT 2026

OpenAI’s self-serve fine-tuning is closing

From OpenAI’s API deprecations page. Fine-tuning on an open model (for example with LoRA) remains an option on your own GPU.

